Expert Systems With Applications 277 (2025) 127251

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Outsmarting algorithms: A comparative battle between Reinforcement Learning and heuristics in Atari Tetris

Julius A. Bairaktaris [a], Arne Johannssen [b,*]

[a] Faculty of Business Administration, University of Hamburg, Moorweidenstrasse 18, Hamburg, 20148, Germany
[b] Faculty of Business Studies, Harz University of Applied Sciences, Friedrichstrasse 57–59, Wernigerode, 38855, Saxony-Anhalt, Germany

ARTICLE INFO

Keywords:
Reinforcement Learning
Heuristics
Tetris
Deep Q-network
Categorical DQN
Proximal policy optimization
Arcade learning environment

ABSTRACT

A comparative analysis of Reinforcement Learning (RL) and heuristic methods in solving the classic Tetris game, specifically using the Atari Learning Environment (ALE) implementation, is presented in this paper. Three state-of-the-art RL algorithms – Deep Q-Network (DQN), Categorical DQN (C51), and Proximal Policy Optimization (PPO) – alongside a custom heuristic agent are examined. Considerable disparities in performance and computational efficiency between these approaches are revealed by our study. The heuristic agent consistently outperformed the RL methods in both achieved score and computational efficiency within the constraints of our experimental setup. The paper provides insights into the fundamental challenges of applying RL to environments with delayed rewards and demonstrates the advantages of well-designed heuristics in specific problem domains. The challenges faced by RL agents in the Tetris environment, including sparse rewards and high-dimensional state spaces, are discussed. The complexities of applying RL to environments with delayed rewards and the potential advantages of well-designed heuristics in specific problem domains are highlighted by our findings. This research therefore contributes to the ongoing discourse on the applicability and limitations of different AI approaches in complex, real-time decision-making scenarios.

1. Introduction

The study of computer game-solving is as old as computer science itself, with chess being one of the earliest and most prominent examples. As the field has evolved, researchers have continually used games to improve and push the boundaries of Artificial Intelligence (AI) and computational problem-solving. This progression has led to significant milestones, from IBM's Deep Blue defeating world chess champion Garry Kasparov in 1997 (Campbell, Hoane, & Hsu, 2002) to more recent achievements like AlphaGo's victory over Go world champion Lee Sedol in 2016 (Chouard, 2016). These accomplishments not only demonstrate the advancing capabilities of AI but also provide valuable insights into problem-solving strategies that can be applied to real-world challenges beyond gaming.

In recent years, the focus of AI research has shifted towards more complex and dynamic environments. Modern state-of-the-art (SOTA) methods in Machine Learning (ML) frequently employ the Atari 2600 and its games to showcase their performance. These studies often utilize the Arcade Learning Environment (ALE), introduced by Bellemare, Naddaf, Veness, and Bowling (2013) in 2013 as a platform for AI research with Atari 2600 games, providing a standardized framework for testing and evaluation. An influential work in this area was published by Mnih et al. (2015), where the ALE was used to demonstrate a DQN learning directly from high-dimensional inputs in Atari 2600 games. This methodology has become increasingly popular, as evidenced by cutting-edge methods such as Categorical DQN (C51), which has demonstrated impressive results across a wide range of Atari games in the ALE (Bellemare, Dabney, & Munos, 2017). By using these classic video games as benchmarks, researchers can evaluate AI algorithms in stochastic environments that demand rapid decision-making, pattern recognition, and adaptability.

Among these classic games, Tetris stands out as a particularly interesting subject of study. The game's combination of real-time decision-making, long chain of actions to get a reward, and theoretically infinite gameplay makes it an ideal testbed for various approaches. This paper provides a comparative analysis of two distinct methods for solving Tetris: Reinforcement Learning (RL) and heuristic approaches. Each of these methodologies represents a different approach in AI and computational problem-solving. RL, a subset of ML, allows agents to learn optimal strategies through trial and error, mimicking the way humans and animals learn from experience. Heuristic approaches employ

* Corresponding author.
E-mail addresses: julius@bairaktaris.de (J.A. Bairaktaris), ajohannssen@hs-harz.de (A. Johannssen).
URL: http://arnejohannssen.com (A. Johannssen).

https://doi.org/10.1016/j.eswa.2025.127251
Received 21 September 2024; Received in revised form 5 February 2025; Accepted 11 March 2025
Available online 19 March 2025
0957-4174/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Table 1
Overview of related work in Tetris AI and RL.

Year	Method	Key contributions	Performance/Notes
1997	IBM's Deep Blue (Campbell et al., 2002)	First major AI victory in chess	Defeated world champion Garry Kasparov
2013	ALE Introduction (Bellemare et al., 2013)	Introduced standardized framework for AI research with Atari 2600 games	Enabled systematic comparison of RL algorithms
2015	DQN (Mnih et al., 2015)	First successful deep RL implementation for Atari games	Achieved human-level performance in multiple Atari games
2016	AlphaGo (Chouard, 2016)	Advanced deep RL in complex game environments	Defeated Go world champion Lee Sedol
2017	C51 (Bellemare et al., 2017)	Introduced distributional perspective to value function approximation in RL, modeling entire distribution of returns rather than just expected values	Demonstrated significant performance improvements across various Atari games through more robust and stable learning
2017	PPO (Schulman, Wolski, Dhariwal, Radford, & Klimov, 2017)	Introduced clipped surrogate objective for policy optimization, combining simplicity of implementation with robust performance	Demonstrated strong performance with improved training stability
2020	SuperStacker (hexhowells, 2022)	Simple heuristic approach for NES Tetris	Achieved score of 4,243,879 points in NES Tetris
2024	BetaTetris (Wu, 2024)	Brute force approach using tablebases	Currently most advanced Tetris AI approach

problem-specific rules and shortcuts to find satisfactory solutions efficiently, often trading optimality for speed. By comparing these methods using Tetris, the goal is to demonstrate the strengths, weaknesses, and applicability to complex, real-time decision-making problems.

Prior to presenting the contributions of this paper, we provide a comprehensive overview of relevant publications in the field of Tetris AI and RL in Table 1. This summary highlights the evolution of approaches and helps contextualize our choice of algorithms.

The overview in Table 1 demonstrates that while DQN, C51, and Proximal Policy Optimization (PPO) may not be the absolute latest SOTA algorithms, they represent significant milestones in RL development and remain highly relevant for comparative studies. Our selection of these algorithms was based on their historical importance, demonstrated effectiveness across various domains and their potential for providing insights into the fundamental challenges of applying RL to Tetris.

The main contributions of this paper are:

- A comprehensive comparison of RL and heuristic methods in solving Atari Tetris in the ALE, demonstrating that heuristics outperform RL in terms of performance and computational efficiency.
- An analysis of the performance and computational efficiency of different RL algorithms (DQN, C51, PPO) in the Tetris environment, showing that while C51 offers advantages over DQN, modern SOTA methods still struggle to master Tetris within reasonable computational limits.
- An exploration of the challenges faced by RL agents in environments with sparse rewards and high-dimensional state spaces, highlighting the difficulties in Tetris due to its inherent stochasticity and the need for precise image recognition.
- A discussion on the trade-offs between the adaptability of RL and the efficiency of heuristic approaches in game-playing scenarios, including the limitations of reward shaping and the advantages of heuristics in transfer learning scenarios and explainability.
- A detailed analysis of the limitations of the ALE/Tetris-v5 environment for AI research, particularly highlighting how its differences from standard Tetris implementations affect agent performance and learning capabilities.

The paper is structured as follows. First, a theoretical background on the two methods discussed is provided in Section 2. Afterwards, in Section 3, Tetris is presented as a computational challenge, discussing its rules, mechanics, and the unique problems it poses for AI. Then, a detailed overview of the implementations of the agents is given in Section 4, followed by an evaluation of their performance in Section 5. Finally, this paper concludes with a discussion on the findings and potential directions for future research in Section 6.

2. Methodological background

This section presents the methodological foundations necessary for understanding the approaches used in this research. We begin by examining RL, a machine learning paradigm that enables agents to learn optimal behaviors through interaction with an environment. We explore both traditional RL methods and their integration with Deep Learning (DL) techniques, focusing particularly on key algorithms like DQN, Categorical DQN (C51), and PPO. Following this, we discuss heuristic methods, which provide an alternative approach to problem-solving through predefined rules and strategies. The section concludes with a comparative analysis of RL and heuristic approaches, highlighting their respective strengths, limitations, and applicability to complex decision-making problems.

2.1. Reinforcement learning

RL is a category of ML that addresses the challenge of learning control strategies to interact with an environment. Drawing inspiration from human and animal learning, RL utilizes knowledge acquired through experience in conjunction with positive and negative reinforcement to achieve specific goals within an environment (Bai, Cheng, & Jin, 2023; Schultz, 2004). This approach has demonstrated remarkable success across various complex domains, from mastering board games like Go (Silver et al., 2017) to excelling in real-time strategy games like StarCraft II (Vinyals et al., 2019), and even navigating stratospheric balloons (Bellemare et al., 2020) as well as Industry 4.0 relevant applications such as the Flexible Job Shop Scheduling (FJSP) problem (Luo, 2020) and cybersecurity applications such as Intrusion Detection Systems (IDSs) (Lopez-Martin, Carro, & Sanchez-Esguevillas, 2020).

A prime example of RL's capabilities is AlphaZero, a groundbreaking algorithm developed by DeepMind. AlphaZero achieved superhuman performance in chess, shogi, and Go without any prior domain knowledge beyond the game rules (Schrittwieser et al., 2020). This success demonstrated the potential of RL combined with Deep Neural Networks (DNNs) and self-play to surpass traditional game-specific approaches, marking a significant milestone in AI research.

The fundamental components of RL include several key elements (Brunton, 2021; Ding, Huang, Yuan, & Dong, 2020):

- Agent: The autonomous entity that learns, makes decisions, and interacts with the environment.
- Environment: The external system or context within which the agent operates and learns.
- State: A comprehensive representation of the environment's current condition, commonly denoted as s.
- Action: A decision or choice made by the agent, typically represented as a.
- Reward: A feedback signal, r, provided by the environment to for the agent's actions.
- Policy: A decision-making strategy, often denoted as π, that maps states to probabilities of selecting each possible action.
- Value function: A function that estimates the expected cumulative discounted reward from a given state, usually denoted as V.

These components work together to form the framework within which the RL agent learns and improves its performance over time.

The primary objective in RL is to find an optimal policy that maximizes the cumulative reward over time. This is typically achieved through iterative improvement of the policy based on experiences gained from interacting with the environment (Brunton, 2021; Carapuço, Neves, & Horta, 2018; Ding et al., 2020).

A key challenge in RL is the Credit Assignment Problem (CAP), which arises from the often sparse and delayed nature of rewards. This makes it difficult for the agent to associate specific actions with their long-term consequences. Thus, RL can be classified as semi supervised learning, since the reward is commonly a time-delayed label (Brunton, 2021; Minsky, 1961; Park et al., 2022).

To address the CAP, reward shaping is often employed to make the rewards more dense. While dense rewards offer higher sample efficiency compared to sparse rewards, they can lead to myopic behavior in the agent, where it learns to maximize short-term rewards at the expense of long-term goals (Dann, Mansour, Mohri, Sekhari, & Sridharan, 2022; Ng, Harada, & Russell, 1999).

The impact of myopic behavior varies depending on the reward structure. In some cases, when rewards are structured to guide the agent towards the goal state, myopic behavior may not be problematic and may even be beneficial to the learning process. However, issues arise when rewards inadvertently mislead the agent away from the goal state.

Moreover, while reward shaping can help guide the agent towards desired behaviors, it can also potentially limit the agent's performance by incorporating pre-existing domain knowledge that might be suboptimal. This reliance on human-designed reward structures may prevent the agent from discovering novel, potentially superior strategies that lie outside the scope of the designer's understanding or expectations (Brunton, 2021; Dann et al., 2022; Ng et al., 1999).

RL can be broadly categorized into two main approaches (Muduli, Jena, & Moger, 2024; OpenAI, 2018):

- Model-based RL: The agent uses a model of the environment for planning.
- Model-free RL: The agent learns directly from experience without having a guiding model of the environment.

Model-free RL can further be categorized into on-policy and off-policy methods. The key difference between these approaches is that on-policy methods take into account the actual behavior of the agent, including potential suboptimal actions due to exploration (Muduli et al., 2024), while an off-policy approach learns from data collected by any policy, including previous versions of the policy or even human demonstrations. This allows off-policy methods to reuse past experiences and learn from observations of other agents, making them potentially more sample-efficient but often less stable during training. This difference can be illustrated through a cliff walking scenario, where on-policy methods would prefer the safer path away from the edge, as it considers the risk of falling off the cliff during exploration, while off-policy would favor the shorter but riskier path just at the edge (Ding et al., 2020). Consequently, the variance in the reward per episode is typically higher in off-policy than in on-policy approaches (Brunton & Kutz, 2022).

Popular RL algorithms such as Q learning and State Action Reward State Action (SARSA) have shown effectiveness in smaller problem spaces. However, traditional RL approaches often struggled with the "curse of dimensionality" – the exponential increase in complexity as the number of state variables grows (Bellman & Dreyfus, 2010). This limitation poses significant challenges when applying RL to real-world problems with large state spaces, such as image-based environments or complex control tasks (Soler, Mariño, Huergo, de Frutos, & Ferrer, 2024).

In recent years, the integration of DL with RL has led to remarkable breakthroughs, addressing these limitations and expanding the applicability of RL across various domains. This combination of techniques has led to Deep Reinforcement Learning (DRL), a powerful approach that combines the representational capacity of DNNs with the decision-making capabilities of RL.

One of the pioneering algorithms in DRL is the DQN, which demonstrated the ability to learn and excel at a wide range of Atari games directly from pixel inputs (Mnih et al., 2015). DQN uses a Neural Network (NN) to approximate the optimal action-value function (1).

$$Q^*(s,a) = \mathop{\mathbb{E}}_{s'\sim P}\left[r(s,a) + \gamma \max_{a'} Q^*(s',a')\right] \tag{1}$$

This is achieved by parameterizing the action-value function with NN weights θ at iteration i (Brunton, 2022; Brunton & Kutz, 2022; Mnih et al., 2015):

$$Q(s,a) \approx Q(s,a,\theta_i) \tag{2}$$

This allows us to optimize for the parameter θ, which has lower dimensionality, in contrast to maximizing the reward for an action a given the state s. To address the instability issue in Q-learning, DQN uses a biologically inspired mechanism called experience replay. Experience replay is based on the complementary learning systems theory, which posits that the hippocampus rapidly learns from new experiences while the neocortex slowly integrates knowledge over time (McClelland, McNaughton, & O'Reilly, 1995). In DQN, experience replay stores the agent's experience (S_t, A_t, R_t, S_{t+1}) at each time step t into a replay buffer. A random small sample is then drawn from the replay buffer to apply the Q-learning update (Brunton, 2022; Dong, Ding, & Zhang, 2020).

The Q-learning update at iteration i uses the following loss function, which utilizes the square of the temporal difference error $R_\Sigma - Q(S,A)$ extended by θ:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim U(D)}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_i^-) - Q(s,a;\theta_i)\right)^2\right] \tag{3}$$

$L_i(\theta_i)$ is optimized through backpropagation and Gradient Descent (GD) techniques. The pseudocode for DQN is provided in Algorithm 1 (Dong et al., 2020):

Algorithm 1 DQN

Hyperparameters: replay buffer capacity N, reward discount factor γ, delayed steps C for target action-value function update, ϵ-greedy factor ϵ
Input: empty replay buffer D, initial parameters θ of action-value function Q
Initialize target action-value function $\hat{Q}$ with parameters $\hat{\theta} \leftarrow \theta$
for episode = 0, 1, 2, … **do**
  Initialize environment and get observation O_0
  Initialize sequence $S_0 = \{O_0\}$ and preprocess sequence $\phi_0 = \phi(S_0)$
  for $t = 0, 1, 2, \ldots$ **do**
    With probability ϵ select a random action A_t, otherwise select $A_t = \arg\max_a Q(\phi(S_t), a; \theta)$
    Execute action A_t and observe O_{t+1} and reward R_t
    If the episode has ended, set $D_t = 1$. Otherwise, set $D_t = 0$
    Set $S_{t+1} = \{S_t, A_t, O_{t+1}\}$ and preprocess $\phi_{t+1} = \phi(S_{t+1})$
    Store transition $(\phi_t, A_t, R_t, D_t, \phi_{t+1})$ in D
    Sample random minibatch of transitions $(\phi_i, A_i, R_i, D_i, \phi'_i)$ from D
    If $D_i = 0$, set $Y_i = R_i + \gamma \max_{a'} \hat{Q}(\phi'_i, a'; \theta^-)$. Otherwise, set $Y_i = R_i$
    Perform a GD step on $(Y_i - Q(\phi_i, A_i; \theta))^2$ with respect to θ
    Synchronize the target $\hat{Q}$ every C steps
    if the episode has ended **then**
      break
    end if
  end for
end for

Various alternative algorithms to DQN have been developed to address its limitations. For instance, Double Deep Q Network (DDQN) reduces the overestimation present in DQN (van Hasselt, Guez, & Silver, 2016; Shakya, Pillai, & Chakrabarty, 2023). Another notable variant is the Categorical DQN (C51), which addresses the limitation of representing the Q-value as a single point estimate. Instead, C51 models the entire distribution of possible Q-values for each state–action pair. This is achieved by discretizing the Q-value range into a fixed number of atoms (typically 51, hence the name C51) and learning a probability distribution over these atoms (Bellemare et al., 2017).

The value distribution for a state–action pair (s, a) is then represented as:

$$Z_\theta(s,a) = z_i \quad \text{w.p.} \quad p_i(s,a) := \frac{e^{\theta_i(s,a)}}{\sum_j e^{\theta_j(s,a)}} \tag{4}$$

To overcome the challenges faced by working with a discrete value distribution, namely the restriction of having to utilize sample transitions, which is not possible under the Wasserstein loss, the Bellman update can be reduced to multiclass classification,

$$(\Phi\tilde{T}Z_\theta(s,a))_i = \sum_{j=0}^{N-1}\left[1 - \frac{|\tilde{T}z_j|_{V_{\min}}^{V_{\max}} - z_i}{\Delta z}\right]_0^1 p_j(s', \pi(s')), \tag{5}$$

where N is the number of atoms in the discrete distribution, z_j is the jth atom value, $V_{\min}$ and $V_{\max}$ are the lower and upper bounds of the distribution support respectively, Δz is the atom spacing defined as $(V_{\max} - V_{\min})/(N-1)$, s' is the next state, and $p_j(s', \pi(s'))$ is the probability of the jth atom in the next state s' under policy π (Bellemare et al., 2017).

The sample loss function for C51 is the cross-entropy of the Kullback Leibler (KL) divergence (Bellemare et al., 2017):

$$D_{\mathrm{KL}}(\Phi\tilde{T}Z_\theta(s,a) \parallel Z_\theta(s,a)) \tag{6}$$

The C51 algorithm has shown remarkable performance improvements over traditional DQN in various Atari games (Tetris excluded), as shown in Table 2. Its ability to learn distributions contributes to more stable and robust learning in inherently stochastic environments.

The pseudocode for the C51 algorithm is as follows Bellemare et al. (2017):

Algorithm 2 Categorical Algorithm (C51)

Require: A transition $x_t, a_t, r_t, x_{t+1}, \gamma_t \in [0, 1]$
$Q(x_{t+1}, a) := \sum_i z_i p_i(x_{t+1}, a)$
$a^* \leftarrow \arg\max_a Q(x_{t+1}, a)$
$m_i = 0, \quad i \in 0, \ldots, N-1$
for $j \in 0, \ldots, N-1$ **do**
  $\tilde{T}z_j \leftarrow [r_t + \gamma_t z_j]_{V_{\min}}^{V_{\max}}$ ▷ Compute the projection of $\tilde{T}z_j$ onto the support $\{z_i\}$
  $b_j \leftarrow (\tilde{T}z_j - V_{\min})/\Delta z$ ▷ $b_j \in [0, N-1]$
  $l \leftarrow \lfloor b_j \rfloor, u \leftarrow \lceil b_j \rceil$
  $m_l \leftarrow m_l + p_j(x_{t+1}, a^*)(u - b_j)$ ▷ Distribute probability of $\tilde{T}z_j$
  $m_u \leftarrow m_u + p_j(x_{t+1}, a^*)(b_j - l)$
end for
return $-\sum_i m_i \log p_i(x_t, a_t)$ ▷ Cross-entropy loss

PPO provides another significant advancement in DRL algorithms, addressing some of the limitations of previous on-policy methods. Introduced by Schulman et al. (2017), PPO aims to improve the stability and sample efficiency of policy optimization while maintaining simplicity of implementation.

PPO uses a clipped surrogate objective function to ensure that policy updates are not too large, which helps with instability during training. The PPO objective function is defined as

$$L^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)], \tag{7}$$

where $r_t(\theta)$ is the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}$, $\hat{A}_t$ is an estimator of the advantage function at timestep t, and ϵ is a hyperparameter (typically 0.2) that controls the clipping range. The clipping in the objective function serves to mitigate updates to the policy function that would change it significantly.

PPO also introduces the concept of multiple epochs of minibatch updates. This allows the algorithm to reuse each collected sample multiple times, improving sample efficiency. The pseudocode for PPO is as follows Schulman et al. (2017):

Algorithm 3 Proximal Policy Optimization (PPO)

for iteration = $1, 2, \ldots$ **do**
  for actor = $1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\mathrm{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\mathrm{old}} \leftarrow \theta$
end for

In comparison to DQN and C51, PPO operates in the policy space rather than the value space. While DQN and C51 focus on learning value functions (with C51 learning a distribution over values), PPO directly optimizes the policy. This approach can be particularly effective in continuous action spaces or in environments where the optimal policy is easier to approximate than the optimal value function. Additionally, since it is an on-policy method, it takes its own tendency to explore into consideration and therefore shows less fluctuation in the reward per episode in high-stake environments.

PPO's performance has been demonstrated across various benchmarks, including complex robotic control tasks and Atari games. In

Table 2
Performance summary of C51 and other algorithms on 57 Atari games: Average and median scores relative to human benchmark (Bellemare et al., 2017) (H.B. by Nair et al. (2015)).

Algorithm	Mean	Median	>H.B.	>DQN
DQN	228%	79%	24	0
DDQN	307%	118%	33	43
DUEL.	373%	151%	37	50
PRIOR.	434%	124%	39	48
PR. DUEL.	592%	172%	39	44
C51	**701%**	**178%**	**40**	**50**

many cases, it has achieved SOTA results while being simpler to implement and tune than many competing algorithms (Schulman et al., 2017).

GD, referenced in line 15 of Algorithm 1, is a popular method in ML for finding minima in high-dimensional equations. The vanilla form of GD is called batch GD. Batch GD uses the entire training dataset to compute the gradient of the cost function with respect to the parameters in each iteration. GD consistently converges to global minima at a linear rate, given that initial weight matrices are approximately balanced and the initial end-to-end matrix has a positive deficiency margin. However, calculating the gradient for the entire dataset can be computationally expensive, resulting in poor performance for large datasets. Another challenge is selecting an appropriate learning rate η. A small learning rate can lead to slow convergence, while a large one can hinder convergence and cause fluctuations in the loss function (Arora, Cohen, Golowich, & Hu, 2018; Ruder, 2017).

Stochastic Gradient Descent (SGD) addresses the computational expense of GD by performing parameter updates for each training sample rather than the entire dataset. This enables SGD to learn online. However, SGD's objective function tends to fluctuate due to the use of individual examples (Ruder, 2017).

To mitigate the fluctuation problem of SGD, a momentum term γ (usually set to 0.9 or similar) can be introduced, where the velocity $v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$ is calculated using the previous velocity v_{t-1}, learning rate η, and gradient $\nabla_\theta J(\theta)$, and the parameters θ are updated as $\theta = \theta - v_t$, accelerating SGD in the relevant direction and dampening oscillations.

To address the fixed learning rate issue in both GD and SGD, Adaptive Moment Estimation (Adam) can be employed. Adam combines momentum with an adaptive learning rate, significantly improving robustness, especially with sparse gradients. Unlike previous algorithms where parameters θ were updated with a fixed learning rate η, Adam calculates adaptive learning rates for different parameters using estimates of first and second moments of the gradients (Kingma & Ba, 2015; Ruder, 2017):

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\nabla_{\theta_t} J(\theta_{t,i}) \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2)(\nabla_{\theta_t} J(\theta_{t,i}))^2 \end{aligned} \tag{8}$$

Here, m_t and v_t are estimates of the first moment (mean) and second moment (uncentered variance) of the gradients, respectively, with β_1 and β_2 as decay rates. To counteract the initial bias towards zero, Adam uses bias-corrected versions of these moments (Kingma & Ba, 2015; Ruder, 2017):

$$\begin{aligned} \hat{m}_t &= \frac{m_t}{1-\beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1-\beta_2^t} \end{aligned} \tag{9}$$

The parameters are then updated using these bias-corrected moments:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t \tag{10}$$

The original paper by Kingma and Ba (2015) proposes default values of $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

Nesterov-accelerated Adaptive Moment Estimation (NAdam) extends Adam by incorporating nesterov accelerated gradient, aiming to further improve convergence. However, NAdam is more computationally expensive than the traditional momentum approach in Adam. Adam and NAdam exhibit similar validation errors across various models and datasets, making Adam a favorable choice due to its higher computational efficiency. Adam has become the de facto standard in most DL and DRL applications for the reasons discussed above (Bose & Kumar, 2022; Choi et al., 2019; Houssein, Hassaballah, Ibrahim, AbdElminaam, & Wazery, 2022).

2.2. Heuristic methods

Heuristics are problem-solving techniques that employ practical, often simplified strategies to find satisfactory solutions efficiently, used when exhaustive search methods are impractical. In the context of AI and problem-solving, heuristics can be defined as rules of thumb or educated guesses that reduce the search space and computational complexity of finding a solution. These methods prioritize speed and practicality over guaranteed optimality, making them particularly valuable in complex problem domains where optimal solutions may be computationally infeasible (Korf, 1990).

Heuristics typically introduce a bias towards certain solution types, as predefined by their developers. This bias can limit the exploration of alternative problem-solving approaches. The effectiveness of heuristic-based solutions is heavily dependent on the quality of the heuristics employed, similar to the "Garbage in, Garbage out" principle in training AI. Poorly designed rules can lead to suboptimal solutions, highlighting the critical role of domain expertise in heuristic design. The quality of the heuristics is often limited by the knowledge available of the environment in which the heuristic will be employed, as well as more general restrictions such as execution time. More complex heuristics, which find better solutions for a given problem, are typically more computationally expensive (Clune, 2007; Korf, 1990).

A significant limitation of heuristics is their problem dependency. Heuristics that work well for one problem may not generalize to other similar problems, often requiring the design of new heuristics for each new problem instance. This lack of generalizability can be particularly challenging in dynamic or evolving problem spaces. Furthermore, heuristics typically follow fixed rules and cannot adapt to unexpected situations or changes in the environment without manual intervention.

In game theory, heuristics play a crucial role in evaluation functions, which assess the quality of a game state and guide decision-making processes. These heuristics identify relevant features of the game state, weigh them according to their importance, and combine them into a single score. This score forms the basis for the decision-making process. The primary challenge in creating game heuristics lies in balancing the extracted features within the combined evaluation function (Clune, 2007).

To address this challenge, Genetic Algorithms (GAs) are frequently employed. This approach begins by initializing a random set of weights for the evaluation function. A fitness function, which in Tetris would be the achieved score, is used to evaluate each set of weights' performance. The GA then applies genetic operators such as selection, crossover, and mutation to evolve the population of weight sets across multiple generations. This iterative process aims to optimize the weights, progressively enhancing the evaluation function (Zhang et al., 2022; Zhou, Tian, Buyya, & Wu, 2023).

2.3. Comparison of reinforcement learning and heuristics

While both RL and heuristic methods aim to solve complex decision-making problems, they differ significantly in their approaches and characteristics. RL algorithms learn optimal policies through direct interaction with the environment, leveraging experience to improve

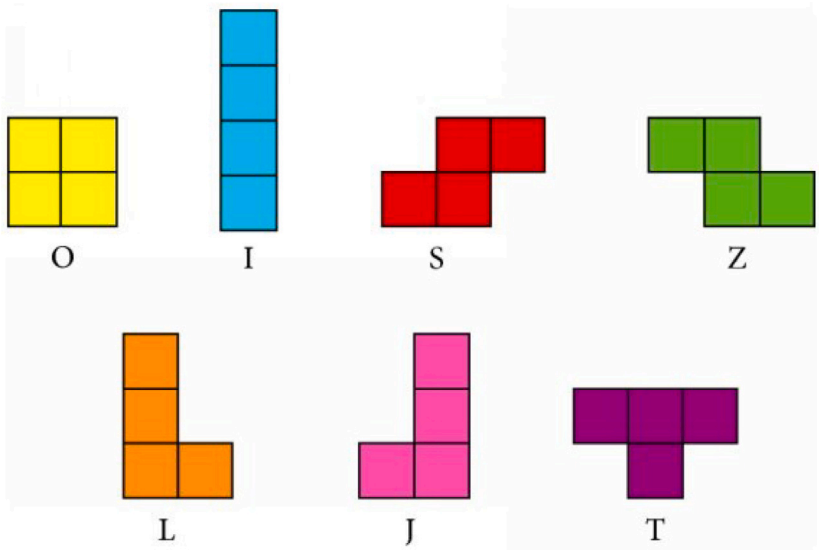

Fig. 1. Naming convention of Tetriminoes (Lewis & Beswick, 2015).

performance over time. This learning-based approach allows RL methods to adapt to changes in the environment and potentially discover novel strategies that may not be apparent to human designers. In contrast, heuristic methods rely on predefined rules designed by domain experts, offering immediate performance but with limited adaptability to new situations.

While the adaptability of RL is a significant advantage, it comes at the cost of increased computational requirements, especially for DRL methods that often demand significant resources for training. Heuristics, on the other hand, typically have lower computational demands during execution, making them more suitable for scenarios with limited processing power or real-time constraints.

Beyond computational considerations, these approaches also differ in their explainability. Heuristic methods are often more interpretable, as their decision-making process follows clear, predefined rules. This transparency can be crucial in applications where understanding the reasoning behind decisions is important. RL policies, particularly those learned by DNN, can be less transparent, often functioning as "black boxes" whose internal decision-making processes are not easily interpreted. While there are methods and approaches to make these black boxes more accessible as outlined by Abusitta, Li, and Fung (2024), the inherent complexity of learned policies can still pose challenges for explainability.

The quest for optimality further distinguishes these methodologies. RL methods aim to find optimal or near-optimal policies through continuous improvement and exploration of the state space. Heuristics, while often effective, may settle for satisfactory solutions that are not necessarily optimal. This difference reflects the fundamental trade-off between exploration and exploitation in decision-making processes.

Closely related to optimality is the ability to generalize. Well-designed RL agents have the potential to generalize to new, similar environments, leveraging learned patterns and strategies. Heuristics, being typically tailored to specific problem instances, may struggle when faced with significantly different scenarios without manual adaptation.

Having explored the theoretical foundations of RL and heuristic methods, we now turn our attention to the specific challenge posed by the game of Tetris. This classic puzzle game serves as an ideal testbed for comparing these approaches, offering a complex, dynamic environment that combines elements of planning, pattern recognition, and real-time decision making.

3. Tetris

3.1. Game rules and mechanics

Tetris, invented by the Russian software engineer Alexey Pajitnov in an Electronika 60 in 1984, is a puzzle video game that has become one of the most recognizable and popular video games worldwide. Despite its seemingly simple core mechanics, Tetris presents a complex challenge for both human players and AI agents. In the years since its creation, numerous versions of Tetris have been released, each introducing new game mechanics. Among these iterations, the 1988 Nintendo Entertainment System (NES) version stands out as particularly influential, significantly shaping the game's evolution and widespread appeal. Since 2001, The Tetris Company has sought to standardize Tetris with the "Tetris Guideline", requiring releases to include a guideline-compliant mode so that anyone with prior Tetris experience can easily adapt to new versions (Tetris Wiki - Tetris Guideline, 2024; The Tetris Company, 2024).

According to the Tetris Guideline, the playfield is 10 cells wide and 20 cells tall, with an additional 20-cell buffer zone above the playfield. This buffer zone is usually fully or partially obstructed by the field frame, depending on the console's hardware limitations.

On the playfield, the player can place pieces known as Tetriminoes that fall from the top. Each Tetrimino consists of four square blocks arranged in different shapes. There are seven distinct Tetrimino shapes, often referred to by letters they resemble: I, O, T, S, Z, J, and L, as shown in Fig. 1 (Tetris Wiki - Tetris Guideline, 2024).

Tetriminoes spawn on the 21st and 22nd row of the playfield, centered or, if necessary, off-centered to the left. They spawn either horizontally or with their flat side down (as is the case for J, L, and T) and immediately start falling downwards. After a Tetrimino has either been placed (through a hard drop) by the player or has rested on the structure at the bottom of the playing field for over half a second, it locks into that position, and a new Tetrimino spawns (Tetris Wiki - Tetris Guideline, 2024).

The spawning of Tetriminoes is defined by the 7-bag Random Number Generator (RNG). This generator creates a sequence of all 7 Tetriminoes randomly, as if drawn from a bag. It then deals from this bag until it is empty, at which point it generates a new bag. This results in 7! possible bags from which the game can choose. According to the Tetris Guideline, the next 1 to 6 pieces must be shown to the player, with 6 being the recommended number. The guidelines also define a "hold" mechanic, allowing the player to store a Tetrimino in

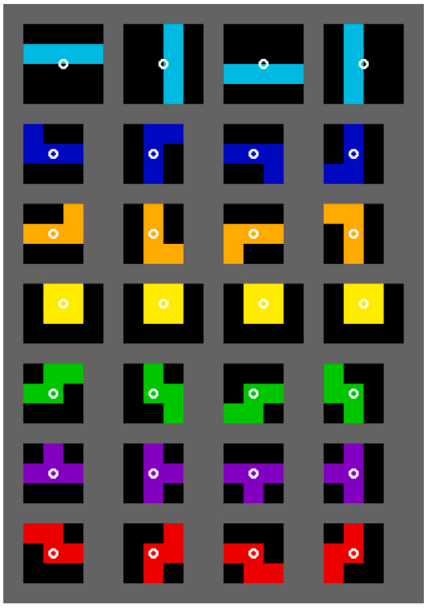

Fig. 2. Tetrimino rotation according to the SRS (Tetris wiki - SRS, 2024).

the hold box and swap it with the Tetrimino already present there. If no Tetrimino is in the hold box, the next piece is chosen. The hold mechanic cannot be used again before the active Tetrimino locks onto the structure (Tetris Wiki - Tetris Guideline, 2024).

The Tetris Guideline defines the SRS as the current standard for how players can manipulate the Tetriminoes. SRS was introduced in its final form in the 2001 game, Tetris Worlds. In the SRS, during the fall of the Tetrimino, the player can (Tetris wiki - SRS, 2024)

- move the Tetrimino left or right,
- rotate it 90 degrees clockwise or counterclockwise,
- accelerate its downward movement (soft drop) or
- instantly drop it to the lowest possible position (hard drop).

When unobstructed, Tetriminoes can be rotated around a single centered point, as shown in Fig. 2. When a rotation is attempted in an obstructed position, five positions are sequentially tested (including the basic rotation), as seen in Fig. 3. If no rotations are legal, the rotation fails completely. A legal rotation is one that does not pass through already present blocks (Tetris wiki - SRS, 2024).

Players score points by clearing lines. A line is cleared when a horizontal line of the playfield is completely filled with blocks, resulting in all blocks above moving down by the number of cleared lines. Clearing multiple lines with one piece results in higher scores, with the highest possible reward given for clearing four lines at once, called a "Tetris". The line values for cleared lines at once are as follows:

- Single = 1 line
- Double = 3 lines
- Triple = 5 lines
- Tetris = 8 lines

As the game progresses, Tetriminoes begin to fall faster, increasing difficulty by forcing the player to make quicker decisions. The game ends when new Tetriminoes cannot enter the playing field after spawning in the center without the structure exceeding the bounds of the observable playing field (Tetris Wiki - Tetris Guideline, 2024).

3.2. Implementation details and limitations of ALE/Tetris-v5

To compare RL and heuristic methods, RL and heuristic agents have been developed for the ALE. The ALE is a platform for AI research currently maintained by the Farama Foundation. It offers the ability to train AI agents on Atari 2600 games. ALE is often combined with the Gymnasium library, which provides a standard Application Programming Interface (API) to communicate between algorithms and environments (Bellemare et al., 2013; Machado et al., 2018; Towers et al., 2024).

ALE emulates the Atari 2600 using the Stella emulator. Stella is an open-source Atari 2600 video computer system emulator that accurately reproduces the hardware behavior of the original console. The ALE provides a rich feature set simplifying AI development. It automatically extracts the game score from the emulator's memory, providing a natural reward signal for the RL agent (Bellemare et al., 2013; Machado et al., 2018; Mott, Anthony, & Team, 2024). The ALE also provides an option for frame skipping, where the agent only performs an action every n frames reducing computational load when emulating the environment, resulting in potentially faster learning. An image of Tetris in the ALE is given in Fig. 4.

The ALE Tetris (ALE/Tetris-v5) implementation also has its downsides as it does not fully follow the Tetris Guideline. ALE/Tetris-v5 is implemented in 6502 assembly language. A significant downside of the Tetris implementation in ALE is the absence of the 7-bag RNG. The Tetriminoes are generated using a simple pseudo-RNG. Pseudo-RNGs are called "pseudo" since, given the seed value, the string of produced

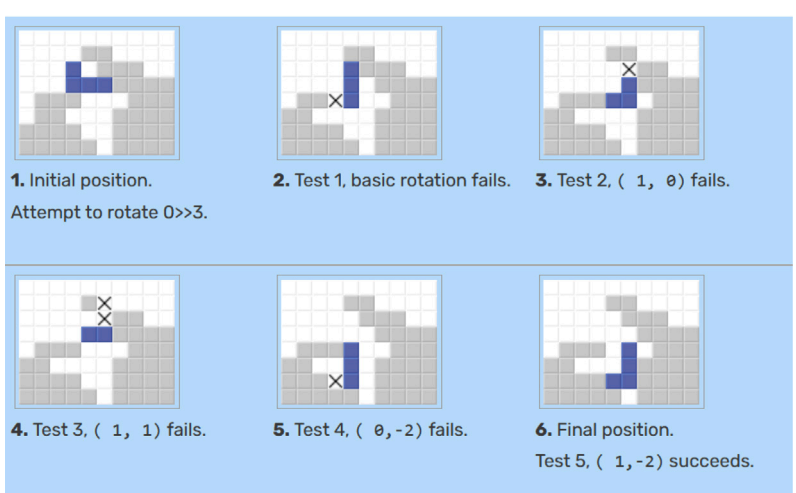

Fig. 3. Wall kick example using the Tetrimino "J" (Tetris wiki - SRS, 2024).

numbers can be determined algorithmically. Specifically, ALE/Tetris-v5's RNG is based on a Linear Feedback Shift Register (LFSR), which is able to generate a random sequence of ones and zeros that are then used to select a random Tetrimino (Bellemare et al., 2013; Machado et al., 2018). The assembly code for the RNG is translated to more readable pseudo-code in Algorithm 4. While the generation of random Tetriminoes is not perfectly random, as more modern and cryptographically secure RNGs would allow for, it still is sufficient to qualify ALE/Tetris-v5 as a Markov Decision Process (MDP) since the next state is only dependent on the present state and action, and every piece has a fixed probability to appear next. This knowledge, however, will not be leveraged in the implementation of the RL agent as it is not present in standard Tetris games that follow the guidelines.

Algorithm 4 Tetris Random Piece Generation

Require: Seed s
Ensure: A random Tetris piece with initial position

```
function RANDOM(s)
    temp ← seed
    if temp = 0 then
        temp ← temp ⊕ 0x1D
    else
        temp ← temp ≪ 1                          ▷ Left shift by 1 bit
        if temp ≠ 0 and CarryFlag = 1 then
            temp ← temp ⊕ 0x1D
        end if
    end if
    seed ← temp                                  ▷ Store the new value
    result ← 0                                   ▷ Initialize the result to 0
    for i ← 0 to 255 do
        result ← result ⊕ Memory[i]              ▷ XOR all memory including the seed
    end for
    return result                                ▷ Return the final pseudo-random number
end function
function RANDOMBLK
    random_value ← RANDOM
    block_index ← random_value ς 0x0F            ▷ Keep only lower 4 bits
    blkid ← blocks[block_index]                  ▷ Select block type from array
    blkx ← 6                                     ▷ Set initial X position
    blkoff ← 21                                  ▷ Set initial Y position
    return
end function
```

In ALE/Tetris-v5, the next piece/pieces are not known, which differs from the Tetris Guideline and makes planning ahead more difficult. In addition, there is no option to hold pieces. Combined with the absence of the 7-bag RNG, playing forever is not possible. To allow the player to play Tetris indefinitely, three conditions must be met (Tetris Wiki - Playing Forever, 2024):

- 7-Bag RNG
- Presence of a Hold feature
- 3 or more pieces are previewed

When these three conditions are met, it is possible for the player to divide the playing field into four left columns, four right columns, and two middle columns. S, T, and Z pieces will be used in the left columns, and L, J, and O will be used in the right columns, while the I pieces will fill the gap in the middle. This strategy would then allow the player to never lose (Tetris Wiki - Playing Forever, 2024).

Another downside is the absence of advanced techniques such as wall-kicking or the 0.5s grace period before Tetriminoes lock into place. The rotation logic is defined under the function `norotate`, which only checks if a rotation is valid and if not, rotates the piece back. It also only allows for clockwise rotations. The locking down of blocks is defined in the functions `blockdown` and `stopped`, which only check if the piece has landed on the structure and if so, generate a new piece with the function described in Algorithm 4.

Table 3
Comparison of ALE/Tetris-v5 and Tetris Guideline.

Feature	ALE/Tetris-v5	Tetris Guideline
Random piece generation	Simple pseudo-RNG (LFSR)	7-bag RNG
Next piece preview	Not available	At least one piece shown, up to 6 pieces shown
Hold feature	Not available	Available
Playing field size	10 × 22	10 × 20
Rotation system	Only clockwise rotation, no wall kicks or other advanced mechanics	SRS
Lock delay	No grace period	0.5-s grace period
Scoring system	1 point per line, 10 points for Tetris	Increasingly more points per line cleared. 100 points for one line
Game speed progression	Constant speed	Increases progressively
Soft drop	Available	Available
Hard drop	Not available	Available
Infinite gameplay	Not possible	Possible with proper strategy

ALE/Tetris-v5 also has a scoring system that differs from the Tetris Guideline. In ALE/Tetris-v5, the reward function $R(L)$ for clearing lines is defined as

$$R(L) = \begin{cases} L, & \text{if } 1 \leq L < 4 \\ 10, & \text{if } L = 4 \end{cases} \tag{11}$$

where L is the number of lines cleared in a single move. This makes the score of the agents difficult to compare to other agents which were developed for Tetris versions that follow the Guideline.

An additional difference from the Tetris Guideline is the absence of incremental game speed progression. In ALE/Tetris-v5, the speed of the falling Tetrimino is determined by the `gamespeed` variable, which is initialized as 16 at the start of the game. The variable is then decreased by 1 for every frame until it reaches 0. If the counter reaches zero, it is reset to the value of `gamespeed` (16), and the game attempts to move the current Tetrimino down.

The Frame Rate is VSync'd to the display's frame rate in the `frame` function. This means that the game either runs at 60 frames per second (FPS) for National Television Standards Committee (NTSC) or 50 FPS for Phase Alternate Line (PAL) systems, which results in a Tetrimino speed of:

- NTSC: $\frac{15}{4}$ Rows/s
- PAL: $\frac{25}{8}$ Rows/s

ALE/Tetris-v5 uses the NTSC format. In ALE/Tetris-v5, only a soft-drop (moving the Tetrimino one row down) but not a hard-drop is present, which negatively impacts the possible score per second. The playing field is also not 10 × 20 as defined by the standard guidelines but 10 × 22 as defined in function `w1` of the game.

An overview of the differences outlined above is given in Table 3.

Additionally, an overview of ALE/Tetris-v5 action mapping is given in Table 4.

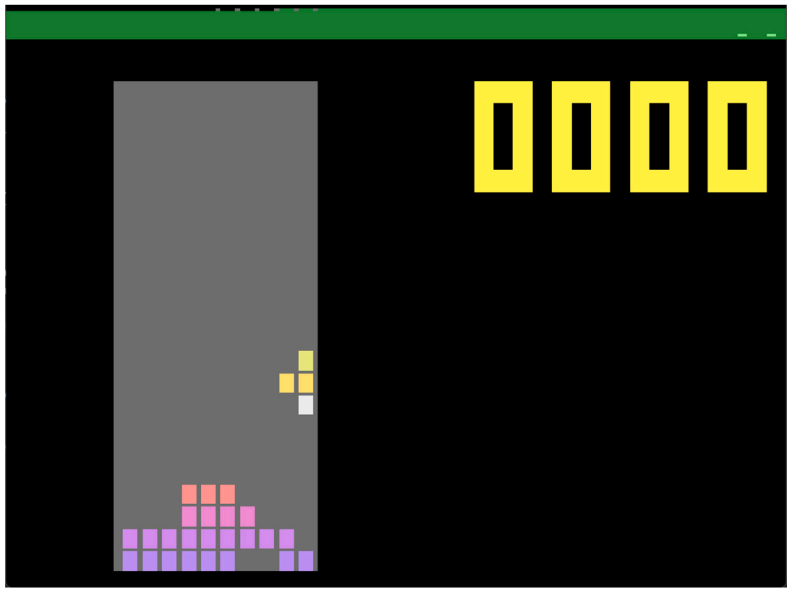

Fig. 4. Screenshot of ALE/Tetris-v5.

Table 4
Action mapping in ALE/Tetris-v5 (Bellemare et al., 2013; Machado et al., 2018; Towers et al., 2024).

Value	Meaning	Value	Meaning	Value	Meaning
0	NOOP	1	FIRE	2	RIGHT
3	LEFT	4	DOWN		

3.3. Tetris as a challenge

Tetris presents a particularly interesting example for ML applications as well as more traditional methods like heuristic and brute force approaches. While it appears simpler on the surface than more studied games like Go and chess, it still poses complex challenges to master. One of the key factors contributing to this complexity is the vast state space combined with stochastic piece generation. The state space in Tetris is $2^{200} \approx 1.61 \times 10^{60}$, since every cell of the 10×20 board can be in one of two states: occupied or empty. The state space of ALE/Tetris-v5 is $2^{220} \approx 1.68 \times 10^{66}$. Additionally challenging is Tetris's high branching factor. Only the possible column positions are relevant to the branching factor as we cannot place pieces under already existing structures, at least in the ALE/Tetris-v5 environment without lock delay. For Tetris, the highest number of possible positions is given for the L, J, T pieces with 4 distinct rotations and a width of either 2 or 3 blocks (depending on the rotation). This results in 2×8 (in the 2 width state) $+ 2 \times 9$ (in the 3 width state) $= 34$ possible positions per piece. The maximum possible moves in one turn for Tetris is 162: 9 possible placements for O, 17 for I, 17 each for Z and S, and the aforementioned 34 possible positions for each of the 3 pieces L, J, and T (Müller-Brockhausen, Preuss, & Plaat, 2021).

Müller-Brockhausen et al. (2021) have empirically shown that the state-space complexity in Tetris Link (a Tetris-style board game where lines do not vanish after being filled) is $37^{74} \approx 1.45 \times 10^{69}$. Therefore, the state-space complexity in Tetris video games where filled lines clear and the game continues for longer is larger than 1.45×10^{69}. For reference, the state-space complexity of chess is less than 4×10^{37} according to (Gourion (2021)).

3.3.1. Tetris as a reinforcement learning challenge

When creating an RL agent, the key challenge is always addressing the CAP. Tetris is an environment with delayed sparse rewards. Multiple actions must be executed in succession to clear a line and, therefore, obtain a reward. In ALE/Tetris-v5, the CAP is further aggravated by the different scoring system in which the incentive to develop strategies that focus on clearing multiple lines with one piece is lower, since only Tetrises are rewarded significantly more. This could potentially lead to suboptimal strategies where the agent exhibits myopic behavior, trying to maximize short-term single line clears over Tetrises.

One of the primary difficulties is the stochastic nature of the next piece. Since the agent does not know which pieces are next in the ALE, planning ahead becomes very challenging. For instance, the placement of a piece given a certain board state could result in a very high reward if the subsequent pieces are favorable for that action. Conversely, the same action for the same board state might yield poor results if unfavorable pieces are randomly generated. This inconsistency in the reward signal for a given sequence of actions can confuse the learning process of the agent.

Another significant challenge to overcome in RL is the tuning of hyperparameters. In the context of Tetris, this process is particularly complex due to the game's high-dimensional state space and delayed reward structure. Hyperparameters such as learning rate, discount factor, and exploration rate as well as NN architecture can dramatically impact the agent's performance. The long episodes and sparse rewards in Tetris make it difficult to quickly assess the impact of hyperparameter changes, often requiring extensive training periods to observe meaningful differences. Moreover, the stochastic nature of piece generation introduces additional variability, potentially obscuring the true effects of hyperparameter or architecture adjustments.

Additionally, balancing exploration and exploitation is particularly challenging in Tetris since one misplaced Tetrimino that results in a significant gap in the structure could lead to the agent being unable to recover from one poor decision and not receive any further rewards. This high-stakes environment complicates the agent's ability to explore different strategies without risking premature game termination, making it particularly difficult for off-policy agents to master.

A unique challenge in ALE/Tetris-v5 is the absence of a hard drop option, which allows for a significant number of redundant inputs. After the agent has decided on a piece's final position and rotation, it must wait for the piece to descend to its intended location. During this waiting period, the agent continues to make decisions, potentially leading to suboptimal or contradictory actions. This extended time between decision and outcome not only increases the difficulty of credit assignment but also provides opportunities for the agent to explore irrelevant or detrimental actions, further complicating the learning

process. The lack of a hard drop option thus extends the temporal gap between action and reward.

Moreover, the visual representation of ALE/Tetris-v5 introduces additional complexities. The agent must learn to distinguish between different piece shapes, colors, and orientations, as well as interpret the current state of the game board with high precise compared to other Atari games like Pong. This visual parsing task is non-trivial and adds an extra layer of complexity to the learning process. The agent also needs to develop an understanding of spatial relationships and how pieces fit together, which is more challenging when working with raw pixel data compared to a more abstract representation of the game state.

Similar to the challenges faced by Montezuma's Revenge, Tetris in the ALE poses significant challenges for agents attempting to learn solely through random exploration. In Montezuma's Revenge, the sparsity of rewards and the need for precise sequences of actions make it extremely unlikely for an agent to stumble upon positive reinforcement signals through random exploration. In experiments conducted by Mnih et al. (2015), the DQN achieved a normalized score of 0% against human testers in Montezuma's Revenge. Therefore, approaches similar to Shen et al. (2024) are often employed, which combine heuristic reward shaping functions similar to those discussed in Section 4.2 in environments with significant temporal gaps such as Tetris to stabilize learning (Huynh, 2020).

3.3.2. Tetris as a heuristic challenge

Heuristic agents face challenges similar to those encountered by RL agents in Tetris, particularly in processing image data as input and dealing with the game's stochastic nature. However, heuristic agents also have their unique challenges.

The primary challenge in developing a heuristic agent for Tetris is balancing the conflict between short-term and long-term objectives. A heuristic that prioritizes immediate line clears might lead to suboptimal board configurations over time and potentially result in lower scores per piece placed. This is particularly significant given that a Tetris awards ten times more points than clearing a single line in ALE/Tetris-v5. Consequently, long-term strategies that prioritize setting up Tetrises are desirable but difficult to develop. Such strategies also risk premature game termination if the structure rises too high while waiting for the necessary pieces, which are not guaranteed within 7 pieces in ALE/Tetris-v5.

Another complex challenge lies in the implementation of advanced techniques commonly used in human expert play. These include wall kicks and utilizing the 0.5-s grace period during which a Tetrimino remains movable after landing on the structure. Incorporating these mechanics into a heuristic function is non-trivial, as they require precise knowledge of the piece's position, which is difficult to implement when working with image data.

4. Methodology

For the comparison between RL and heuristic methods using ALE/Tetris-v5, the CleanRL library by Huang, Dossa, Ye et al. (2022) has been chosen as the basis for the RL agent. The heuristic agent utilizes "SuperStacker" (hexhowells, 2022) as the foundation for its evaluation function. CleanRL has been chosen since it offers high-quality single-file implementation of modern DRL algorithms in JAX and PyTorch. In comparison to other popular libraries such as Tianshou by Weng et al. (2022) or Dopamine by Castro, Moitra, Gelada, Kumar, and Bellemare (2018), does CleanRL provide a more streamlined and modular codebase by using wrappers from Stable Baselines3, making it ideal for research and experimentation (Engstrom et al., 2020).

4.1. Implementation of reinforcement learning

We have chosen DQN and C51 for the off-policy RL agents. DQN was selected to first create a baseline for comparison, as no prior DQNs have been extensively trained for ALE/Tetris-v5. We then chose C51 as a modern variant of DQN to explore potential improvements in performance and robustness over the baseline model. C51 was selected because it is a single extension to the DQN algorithm that introduces a distributional perspective on RL, potentially offering more accurate value estimation and improved learning stability. Additionally, C51 is less prone to errors in its implementation compared to other expansions of DQN, as it is significantly less complex than methods such as Rainbow by Hessel et al. (2018), which utilizes numerous extensions to DQN, while offering similar performance in the ALE (Huang, Dossa, Ye et al., 2022).

We chose PPO for our on-policy RL agent, to also explore the difference in using on-policy compared to off-policy methods for Tetris, as on-policy methods take their own tendency to explore into consideration which could lead to premature termination in high stakes environments such as Tetris (Ding et al., 2020). PPO was chosen specifically for its impressive performance in the ALE compared to other on-policy methods (Huang, Dossa, Ye et al., 2022).

4.1.1. Implementation of DQN

The architecture of the DQN agent consists of two main components: a Convolutional Neural Network (CNN) for processing the game state, and a broader RL framework that manages the learning process. The components of the DQN agent are as follows Huang, Dossa, Ye et al. (2022):

The Tetris environment is wrapped with the `atari_wrappers` from the Stable Baselines3 library (Raffin et al., 2021) using several preprocessing techniques:

- ClipRewardEnv: Clips rewards to $\{+1, 0, -1\}$, normalizing the reward scale.
- EpisodicLifeEnv: Considers the loss of a life as the end of an episode, helping with value estimation.
- FireResetEnv: Executes the FIRE action on reset for environments that are fixed until firing. For ALE/Tetris-v5, this is not the case.
- MaxAndSkipEnv: Returns only every skip-th frame (frameskipping) and provides the maximum pixel values between the two most recent frames.
- NoopResetEnv: Performs a random number of no-operations on reset to sample the initial state. No-operations are assumed to be action "0".

And Gymnasium (Towers et al., 2024) wrappers for additional preprocessing:

- FrameStack: Stacks 4 consecutive frames to provide temporal information.
- GrayScaleObservation: Converts the observation to grayscale to reduce dimensionality.
- ResizeObservation: Resizes the observation to an 84×84 image.

The Q-network is implemented using the Flax library (Heek et al., 2024), an NN library and ecosystem for JAX (Bradbury et al., 2018). The architecture consists of:

- Input Layer: Accepts a stack of 4 grayscale frames, each 84×84 pixels, resulting in $4 \times 84 \times 84$.
- Normalization: Normalizing the input by dividing each pixel value by 255.0. This converts the 8-bit integer pixel values (ranging from 0 to 255) into floating-point numbers between 0 and 1, inclusive.
- Convolutional Layers:
  - Conv2D(32, kernel_size = (8, 8), strides = (4, 4), padding = "VALID")
  - Rectified Linear Unit (ReLU) activation

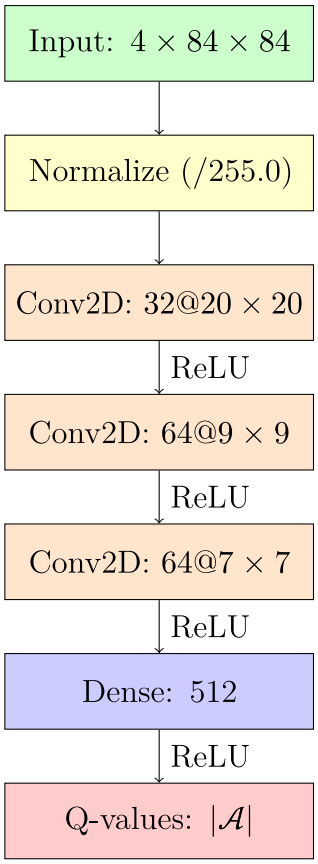

Fig. 5. DQN Architecture for Atari Games as defined in `dqn_atari_jax.py` by CleanRL (Huang, Dossa, Ye et al., 2022).

  - Conv2D(64, kernel_size = (4, 4), strides = (2, 2), padding = "VALID")
  - ReLU activation
  - Conv2D(64, kernel_size = (3, 3), strides = (1, 1), padding = "VALID")
  - ReLU activation

- Flatten operation
- Fully connected layers:
  - Dense(512) with ReLU activation
  - Dense(action_dim) for Q-value outputs

This CNN architecture is responsible for extracting relevant features from the game state and mapping them to Q-values for each possible action. The CNN architecture is depicted in Fig. 5. The output of each layer has been calculated using the formula $[(W-K+2P)/S]+1$, where W is the input width/height, K is the kernel size, P is the padding (0 for "VALID" padding), and S is the stride (Lecun, Bottou, Bengio, & Haffner, 1998).

The RL framework is used for the learning process and includes several components (Huang, Dossa, Ye et al., 2022):

- Replay Buffer: Stores experience tuples (state, next_state, actions, rewards, terminations, infos) with a capacity of 1,000,000 transitions.
- Epsilon-Greedy Policy: Manages exploration–exploitation trade-off, reducing ϵ from 1.0 to 0.01 over 10% of total timesteps.
- Q-Network: Consists of two identical CNN architectures discussed above:
  - Online Network: Updated every 4 steps.
  - Target Network: Updated every 1000 steps using Polyak updates with $\tau = 1.0$
- Optimizer: The Adam optimizer is used with a learning rate of 1×10^{-4}. The rationale for using Adam is given in Section 2.1.

The hyperparameters in CleanRL (Huang, Dossa, Ye et al., 2022) differ from those used by Mnih et al. (2015), as they have been shown to reach similar scores with 1/5 of training steps for Atari games such as Breakout, Pong, and BeamRider. Key differences include the use of the Adam optimizer instead of RMSprop, different learning starts and target network update frequencies, and changes in exploration parameters. A detailed overview of the changes is given in Table 5.

4.1.2. Implementation of C51

The implementation in CleanRL's `c51_atari_jax.py` shares many similarities with the DQN implementation such as the preprocessing but introduces several key differences:

- Network Architecture: Instead of estimating a single Q-value for each action, C51 estimates a discrete probability distribution over a fixed set of atoms (51 in this implementation) representing possible return values. The 51 atoms are linearly spaced between V_min (−10) and V_max (10). The output layer is modified to produce $action_dim \cdot n_atoms$ values, which are then reshaped and passed through a softmax function to obtain valid Probability Mass Functions (PMFs).
- Loss Function: C51 uses the cross-entropy loss between the target and predicted PMF, rather than the MSE loss used in DQN.

The C51 algorithm maintains the same basic structure as the DQN illustrated in Fig. 6, including the use of a replay buffer, target network, and epsilon-greedy exploration. However, the richer representation of the value function allows for potentially more nuanced learning and decision-making.

In comparison to Bellemare et al. (2017), the C51 implementation does not inject stochasticity by rejecting the agent's selection for each frame with a probability of 0.25. Additionally, a self-contained evaluation scheme is used whereas Bellemare et al. (2017) uses a separate evaluation process.

The hyperparameters used are given in Table 6.

4.1.3. Implementation of PPO

The PPO algorithm, implemented in CleanRL's `ppo_atari.py`, also uses the same preprocessing as the DQN and C51 implementations but introduces several key differences due to its on-policy nature. It is based on the "9 Atari implementation details" by Huang, Dossa, Raffin et al. (2022):

- Actor-Critic Architecture: Shared feature extractor based on a CNN for the policy network and value network
- Absence of Replay Buffer: Since it collects experiences using the current policy and updates the policy based on these experiences it does not use a replay buffer.
- Vectorized Environment: A simulation approach where multiple instances of an environment run simultaneously in parallel, enabling the collection of experiences from multiple agents at once. In our implementation, we leverage eight environment simulations running in parallel, significantly reducing wall-clock training time. This technique is particularly effective in RL for game environments (Raffin et al., 2021).
- Clipped Loss: The PPO implementation uses a clipped version of the loss to prevent large policy updates.
- Advantage Estimation: The PPO implementation uses Generalized Advantage Estimation (GAE) to compute advantages, which helps reduce variance in policy gradients.

Table 5
Comparison of DQN Hyperparameters: CleanRL vs. Mnih et al. (2015).

Parameter	CleanRL (`dqn_atari_jax.py`)	Mnih et al. (2015)
Optimizer	Adam (lr = 10^{-4})	RMSprop (lr = 2.5×10^{-4}, momentum = 0.95)
Learning starts	80,000	50,000
Target network frequency	1000	10,000
Total timesteps	10M (40M frames)	50M (200M frames)
Final exploration epsilon	0.01	0.1
Exploration fraction	0.1	0.02
Evaluation scheme	Self-contained	Separate process
Target network updates	Polyak updates (default $\tau = 1.0$)	Hard updates
Loss function	Standard Mean Squared Error (MSE)	Clipped MSE (−1 to 1)

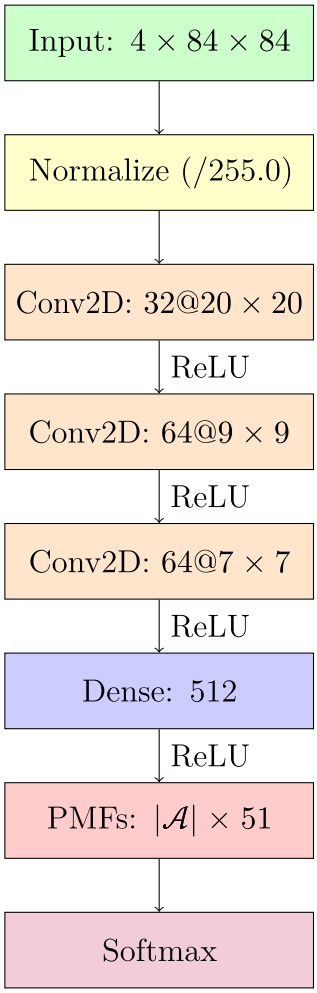

Fig. 6. C51 Architecture for Atari Games as defined in `c51_atari_jax.py` by CleanRL (Huang, Dossa, Ye et al., 2022).

- Multiple Epochs: The algorithm performs multiple optimization steps on the same batch of data, improving sample efficiency.

The PPO implementation maintains a similar CNN architecture to DQN and C51 for processing the game state, as illustrated in Fig. 7. However, it adds separate heads for the actor and critic.

Key hyperparameters for the PPO implementation are given in Table 7:

4.2. Implementation of heuristics

For the heuristic agent, a simple heuristic based on four metrics has been chosen. Advanced techniques such as wall-kicking have not been included in the metrics as they are not present in ALE/Tetris-v5. The agent's score function is based on SuperStacker (hexhowells, 2022). SuperStacker is able to achieve a high score of 4,243,879 points in NES Tetris, its architecture can be found in Fig. 8. For reference, the current world record set by a human player was achieved by Alex Thach with a score of 16,700,760 points using the Tetris Gym practice cart (Contributors, 2024; Hydrant et al., 2024). These scores are directly comparable since both the version used by SuperStacker and the Tetris Gym practice cart implement similar anti-crashing behavior and use the same scoring system, though the significant difference in scores highlights the current gap between heuristic agents and top human players (Contributors, 2024). Prior research has demonstrated the viability of more sophisticated algorithmic approaches in Tetris. Phon-Amnuaisuk (2015) established that GAs integrated with data mining techniques can effectively discover gameplay strategies without explicit rule programming, while enabling direct evolution of gameplay actions rather than merely optimizing predefined evaluation

Table 6
Overview of C51 hyperparameters (Huang, Dossa, Ye et al., 2022).

Parameter	Value
Total timesteps	10,000,000
Learning rate	2.5e−4
Number of parallel environments	1
Number of atoms	51
Return lower bound (V_min)	−10
Return upper bound (V_max)	10
Replay buffer size	1,000,000
Discount factor (gamma)	0.99
Target network update frequency	10,000 timesteps
Batch size	32
Starting epsilon	1
Ending epsilon	0.01
Exploration fraction	0.10
Learning start timestep	80,000
Training frequency	Every 4 timesteps

Table 7
Overview of PPO hyperparameters (Huang, Dossa, Ye et al., 2022).

Hyperparameter	Value
Total timesteps	10,000,000
Learning rate	2.5e−4
Number of parallel environments	8
Steps per environment	128
Learning rate annealing	True
Discount factor (gamma)	0.99
GAE lambda	0.95
Number of minibatches	4
Update epochs	4
Normalize advantages	True
Clip coefficient	0.1
Clip VLoss	True
Entropy coefficient	0.01
Value function coefficient	0.5
Max gradient norm	0.5
Target KL divergence	None

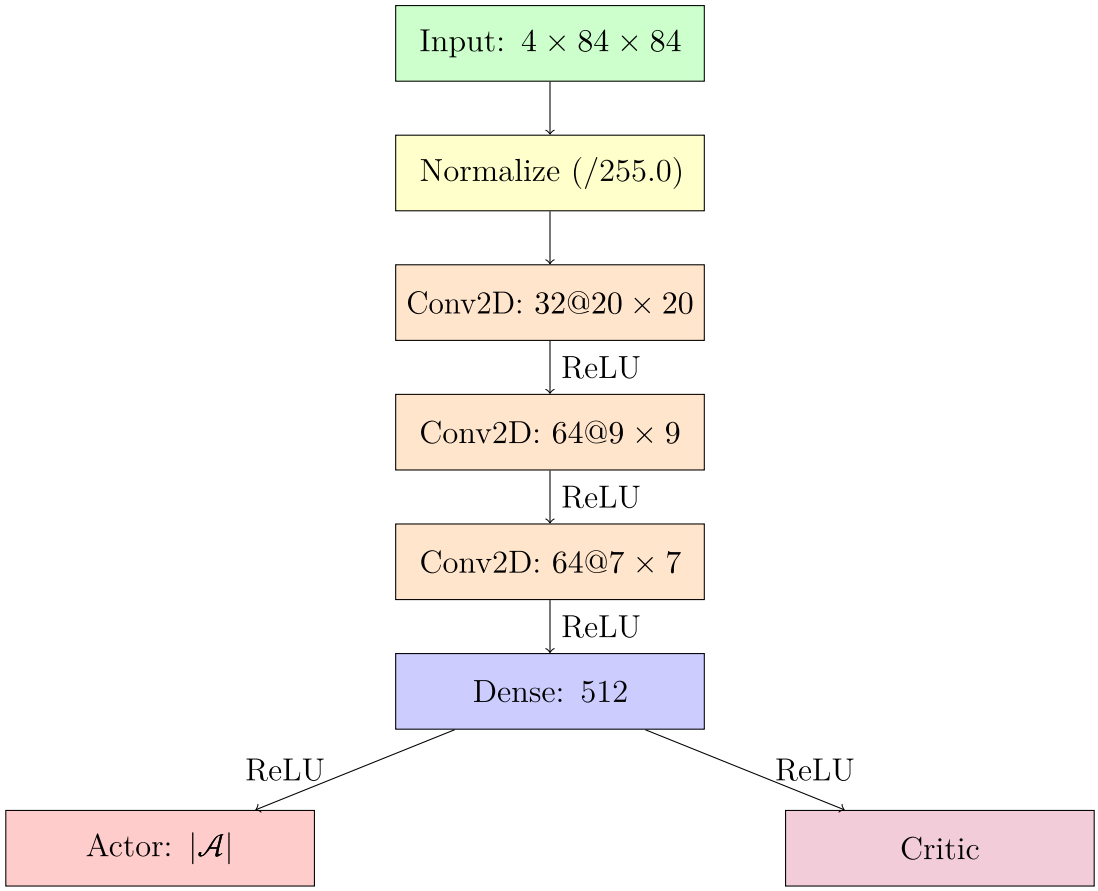

Fig. 7. PPO Architecture for Atari Games as defined in `ppo_atari.py` by CleanRL (Huang, Dossa, Ye et al., 2022).

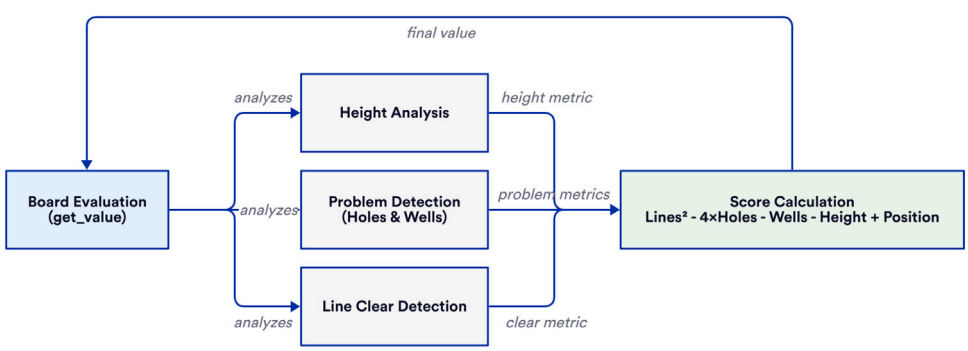

Fig. 8. SuperStacker architecture (hexhowells, 2022).

function weights. Contemporary implementations such as StackRabbit (Cannon, 2024) achieve notable performance through a combination of search algorithms and heuristic methods, utilizing board evaluation via value iteration optimized in C++. However, this research deliberately avoided GA implementation, instead prioritizing the development of a computationally efficient heuristic approach capable of immediate reasonable performance without the extensive training periods characteristic of GAs and ML methodologies. This objective was achieved through strategic utilization of NumPy and Numba for agent optimization.

The agent's decision-making process involves the following steps:

1. Detect the current Tetrimino shape and position.
2. Generate all possible placements for the current Tetrimino.
3. Evaluate each placement using the heuristic function.
4. Select the placement with the highest score; if positions are equal, select the position with a lower column index. Columns are indexed from left to right.
5. Calculate the necessary rotations and movements to achieve the selected placement.
6. Execute the calculated actions.

As we also work with image data in the heuristic agent, we have to perform feature extraction manually without relying on a CNN or similar image detection methods for greater efficiency. The feature extraction process in this implementation is handled primarily by the `preprocess_observation` function. This function takes the raw grayscale image from the ALE and performs several key steps:

1. Cropping: The function first crops the image to a specific region of interest defined by `start_row`, `end_row`, `start_col`,

and `end_col`. This isolates the Tetris playing field from the rest of the screen.

2. Thresholding: The observation space provided by the ALE/Tetris-v5 environment, when set to grayscale mode (`obs_type = ''grayscale''`), is `Box(0, 255, (210, 160), np.uint8)`, representing a 210 × 160 pixel grayscale image with 8-bit depth. The cropped image is converted to a binary image using a threshold value of 111. This step simplifies the image by reducing it to just two values: 0 for empty spaces and 1 for filled blocks. The threshold value of 111 is chosen to be just below the lowest Tetrimino pixel value of 112, ensuring that every Tetrimino is captured while maintaining clear distinction from the background.
3. Downsampling: The binary image is simplified by removing every fourth column and every eighth row. This process eliminates the empty lines between Tetrimino blocks in the ALE/Tetris-v5 environment, as seen in Fig. 4.
4. Resizing: Finally, the image is resized to a 10 × 22 grid, which directly corresponds to the ALE/Tetris-v5 board dimensions.

This preprocessed binary representation of the game state serves as the foundation for Tetrimino detection and identification. The process involves several key steps:

1. Change Detection: The `detect_changes` function compares the current preprocessed image with the image from 16 frames ago. It calculates the difference between these images, identifying newly appeared blocks that represent a Tetrimino. Since Tetriminoes in ALE/Tetris-v5 appear gradually line-by-line, comparing frames separated by 16 frames ensures the complete appearance of the new piece has been captured.
2. Shape Identification: The `detect_tetris_shape` function analyzes the changes detected. It compares the pattern of new blocks against predefined Tetrimino shapes stored in the `tetris_shapes` dictionary. This function considers all possible rotations of each Tetrimino shape. When a match is found, the function returns both the identified Tetrimino shape and its current position (row, column, and rotation) on the board.
3. Structure Extraction: The current game board structure is obtained by subtracting the detected changes from the current binary image, effectively removing the newly appeared Tetrimino.

The four metrics used by the evaluation function are:

- lines_cleared: This metric counts the number of complete lines in the Tetris board. A complete line is one where all cells in the row are filled.
- count_holes: This metric counts the number of empty cells that have at least one filled cell above them in the same column. It is calculated by iterating through each column from top to bottom, keeping track of whether a filled cell has been encountered, and incrementing the hole count for each empty cell found after a filled cell. In Hexhowells' SuperStacker, the `_holes` function considers only empty spaces directly beneath filled cells as holes. When using Hexhowells' approach, the agent was unable to exceed a score of 26.
- count_wells: This metric counts the number of empty cells that have filled cells on both the left and right sides, or are at the edge of the board with a filled cell on the adjacent side. The function iterates through each cell of the board, checking if it is empty (0) and if it has filled neighbors (or board edge) on both sides.
- added_height: This metric measures the increase in the height of the stack after placing a new Tetrimino. It is calculated by finding the difference between the new height and the current height, with a minimum value of 0. This ensures that the added height is never negative, even if the new height is less than the current height.

Table 8
Normalized score distribution statistics.

Score	Heuristic agent
Average	137.50
Std. dev.	40.30
1st percentile	19.00
5th percentile	50.00
10th percentile	67.00
30th percentile	146.00
50th percentile	157.00
70th percentile	157.00
90th percentile	167.00
95th percentile	174.00
99th percentile	190.01

The evaluation function combines these metrics to calculate a score for each possible Tetrimino placement. The score is computed as follows:

$$\text{score} = (\text{lines_cleared}^2) - (\text{holes} \times 4) - \text{wells} - \text{added_height} \tag{12}$$

The quadratic term for lines cleared provides a strong incentive for the agent to aim for multiple line clears when possible, aligning with the scoring system in ALE/Tetris-v5 where a Tetris awards ten times more points than clearing a single line. However, no strategic planning for long-term rewards, e.g., preparing the board for Tetrises, has been implemented. Hexhowells' SuperStacker incorporates an additional term which adds the column index to the score to make the agent prefer stacking towards the right. However, when incorporating this into the ALE/Tetris-v5 agent, the agent was unable to clear over 6 lines per game.

5. Results

Having discussed the implementation of RL and heuristic methods for Tetris, these agents will now be evaluated based on their respective scores achieved as well as metrics specific to each agent. Tables 8 and 9 present the average score, standard deviation and other metrics for the heuristic agent. The evaluation metrics for the RL agents are given in Figs. 10–23. Some plots are specific to the algorithm, therefore not every algorithm is represented in all plots.

5.1. Heuristic agent

For the heuristic agent, performance metrics were generated over 2,000 episodes, each initialized with a random seed. The dataset required 8 h, 21 min, and 52 s to generate on an Intel Core i7-6700K processor clocked at 4.6 GHz. The heuristic agent demonstrated consistent and reliable performance across the 2000 evaluated episodes, achieving an average score of 137.50 with a standard deviation of 40.30. This performance profile reveals several notable patterns (see Table 8).

The score distribution is right-skewed, with the median (157.00) exceeding the mean (137.50). This asymmetry suggests that while the agent typically performs well, occasional poor performances reduce the average. The clustering of scores between the 30th and 70th percentiles (146.00 and 157.00, respectively) indicates a strong core performance band where the agent's strategy is most effective. This tight grouping in the middle percentiles, see Fig. 9, demonstrates the consistency of the agent's decision-making process in typical game situations.

The spread between the 1st percentile (19.00) and the 99th percentile (190.01) reveals the full range of the agent's performance capabilities. The relatively large gap between the 1st and 5th percentiles (19.00 to 50.00) compared to the smaller differences in higher percentiles suggests that catastrophic failures, while rare, can considerably impact the agent's average performance. These extreme low scores likely occur when the agent encounters particularly challenging piece sequences that interact poorly with its heuristic evaluation function.

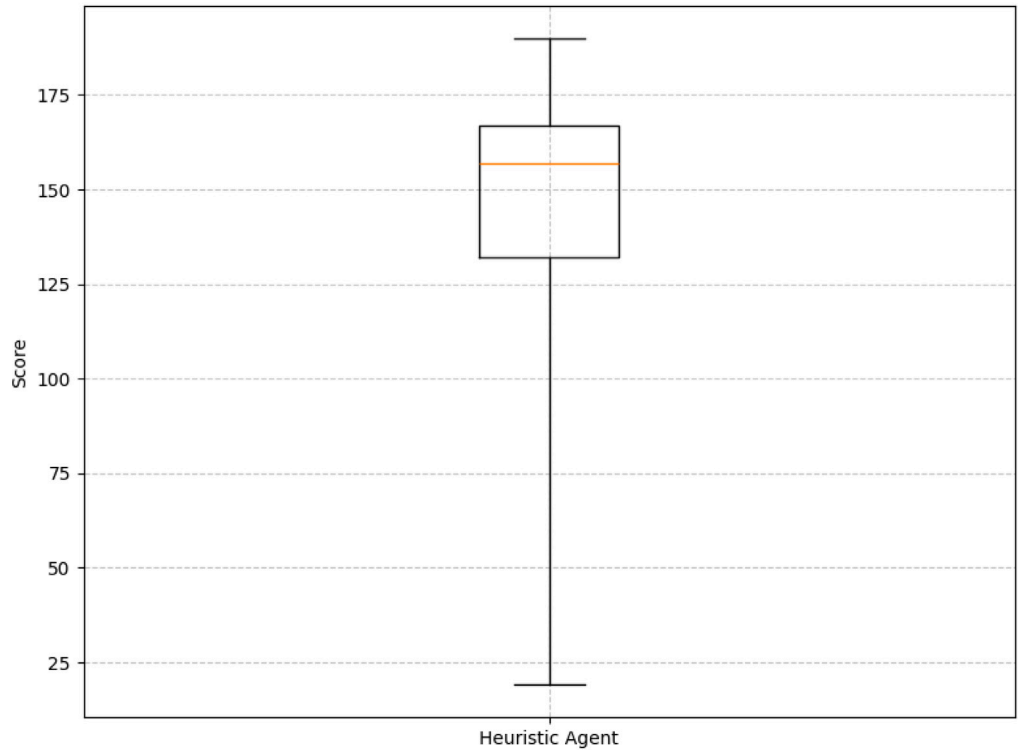

Fig. 9. Boxplot of heuristic agent score.

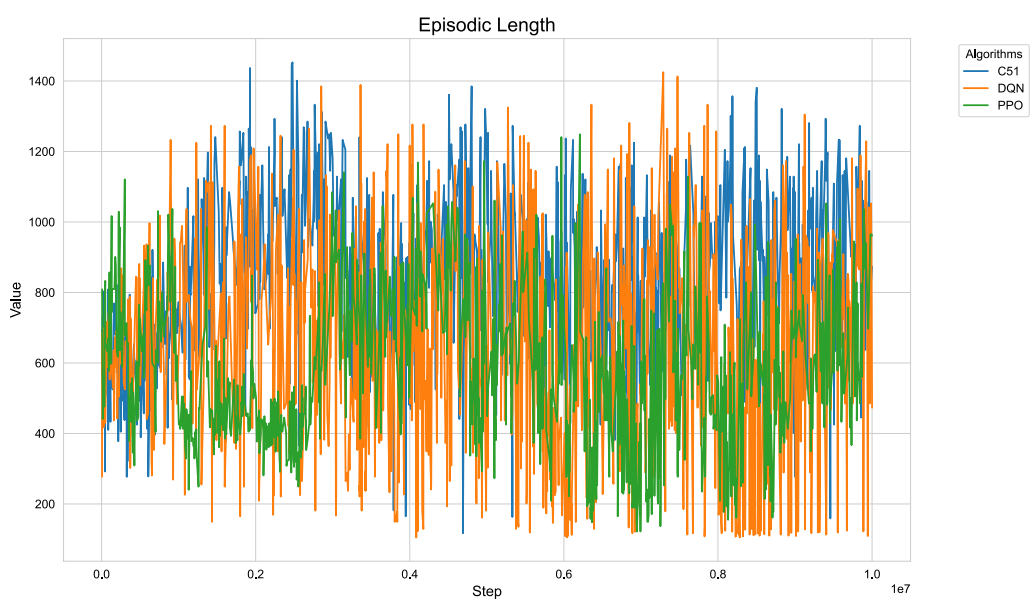

Fig. 10. Episodic length: Shows the duration of training episodes over time, indicating the consistency of the agent's strategy implementation.

The performance stability is particularly evident in the upper percentiles, where the difference between the 90th (167.00) and 95th (174.00) percentiles is relatively small. This compression at the upper end of the performance range suggests that the agent has a practical maximum score limit under its current heuristic framework.

The standard deviation of 40.30 points represents approximately 29.3% of the mean score, indicating moderate variability in performance. This level of variance is reasonable given the stochastic nature of piece generation in ALE/Tetris-v5 and suggests that the agent's heuristic approach maintains reasonable consistency despite the randomness inherent in the game environment.

From a computational efficiency perspective, the agent's ability to process 2000 episodes in approximately 8 h and 22 min translates to an average of 15.1 s per episode. This processing speed is particularly impressive considering it includes the overhead of feature extraction from raw image data. The efficient performance can be attributed to the optimized implementation using NumPy and Numba.

When compared to existing benchmarks, the agent's performance falls considerably short of specialized Tetris agents like StackRabbit or even the SuperStacker implementation it was based on. However, this performance gap must be considered in the context of the limitations and differences in the ALE/Tetris-v5 environment. The absence of

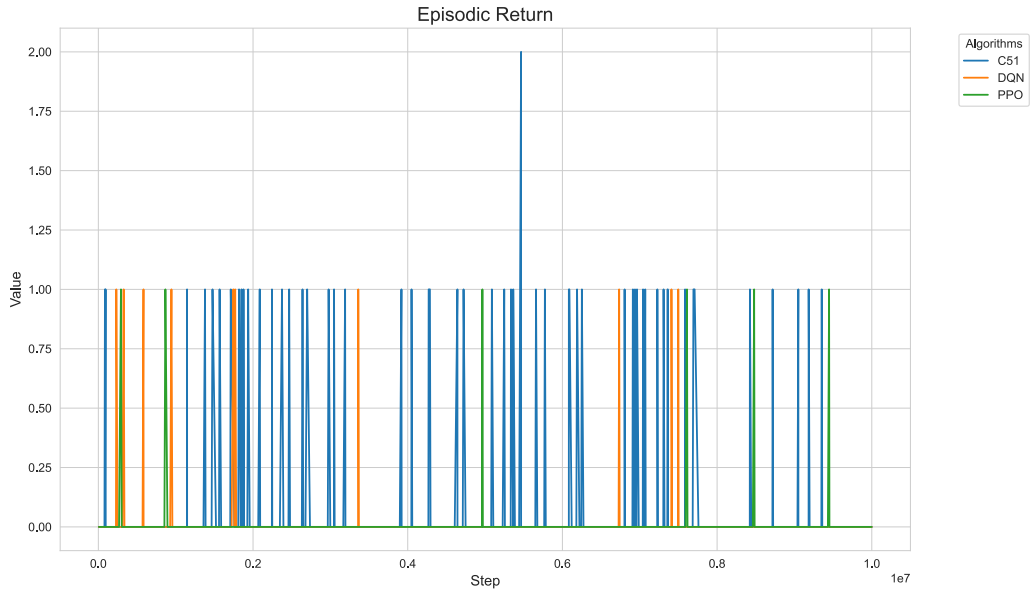

Fig. 11. Episodic return: Demonstrates the cumulative rewards per episode, highlighting the challenges in maintaining stable reward acquisition.

the 7-Bag RNG, as well as having to perform various image processing steps while calculating the optimal tetrimino position according to the heuristic function, creates a fundamentally more challenging environment than those used by other benchmark agents.

5.2. Reinforcement learning agents

In contrast to the heuristic agent, the RL agent's performance metrics reveal a more dynamic and complex learning process. The episodic length in Fig. 10 shows clear fluctuations throughout the training process of all three evaluated algorithms, with no clear trend towards consistently longer episodes. In Tetris, longer episodes typically indicate a better strategy as the agent can survive longer. This volatility suggests that the agent struggles to understand the environment and performs actions without a clear strategy, possibly due to the game's complexity.

5.2.1. C51 agent

C51 achieved the highest average episode length (839.8 steps) and maximum performance (1453 steps) as shown in Fig. 10, while maintaining more consistent positive rewards compared to DQN and PPO (see Fig. 11). At step 5,465,768, the agent cleared two lines with one piece, which was preceded by a cumulative episodic return of 0. This suggests that this event occurred more likely by chance than by a strategy employed by the agent. Nevertheless, C51 was more consistent than DQN or PPO in receiving positive rewards, indicating superiority in the learned policy.

The total loss graph for the C51 agent in Fig. 12 provides further insight into the learning process. The loss starts at a high value of around 4.0 and shows a rapid decline in the early stages of training, indicating strong initial learning. This steep decrease continues until approximately 3 million steps, where the loss stabilizes around 1.0. From this point onward, the loss continues to decrease gradually, reaching 0.5942 by the end of the 10 million steps. This pattern suggests that C51 experiences most of its learning in the early phases, with diminishing returns as training progresses. It is worth noting that despite this apparent stabilization in loss, the agent's gameplay does not show corresponding consistent improvements, underscoring the challenge of translating reduced loss into effective Tetris strategies. The gradual decrease in loss towards the end of training might indicate potential for further improvement with extended training time, though the rate of improvement appears to be slowing and therefore can be considered computationally unfeasible.

5.2.2. DQN agent

The Q-values graph in Fig. 13 allows for insights into the learning dynamics of the DQN- and C51-Agent. C51 demonstrates consistently higher Q-values throughout training compared to DQN, maintaining this superior position across the full training period. The DQN agent exhibits notable instability in its early training phases, characterized by an extreme initial spike in Q-values followed by a sharp decline, clearly indicating value overestimation and subsequent correction. Beyond this point, both agents display a concerning downward trajectory in their Q-value estimates, persisting through the remainder of the training period. Despite C51's more stable Q-value profile and higher estimates compared to DQN, these advantages do not translate into improved gameplay performance, as evidenced by continuing fluctuations in episode duration as shown in Fig. 10. This disconnect between Q-value stability and actual gameplay effectiveness highlights the challenges both agents face in developing reliable strategies for the Tetris environment. The persistent variations in episode length, coupled with declining Q-values, suggest that even with C51's enhanced estimation approach, fundamental difficulties remain in mastering this complex environment.

The Temporal Difference (TD) loss graph shown in Fig. 14 provides further insights into the learning behavior of the DQN agent. The most striking feature is an extreme initial spike in TD loss reaching approximately 0.47 during the early training phase, followed by an immediate and dramatic reduction to near-zero values. This pattern aligns temporally with the Q-value overestimation phenomenon observed in Fig. 13, suggesting a period of substantial policy adjustment as the agent calibrates its value estimates. Following this initial volatility, the TD loss stabilizes at low values, interrupted only by occasional brief spikes at approximately 6, 8, and 10 million steps. These sporadic increases in TD loss, while modest in magnitude compared to the initial spike, coincide with periods of potential policy refinement or environmental challenges. However, the persistent near-zero TD loss throughout most of the training process, combined with the agent's suboptimal performance metrics, suggests that the DQN may have prematurely converged to a suboptimal policy, effectively minimizing its prediction error while failing to discover more effective gameplay strategies.

5.2.3. PPO agent

Fig. 15 shows the learning rate of the PPO agent, which linearly decreases from 2.5×10^{-4} to 0 over the course of training. This allows

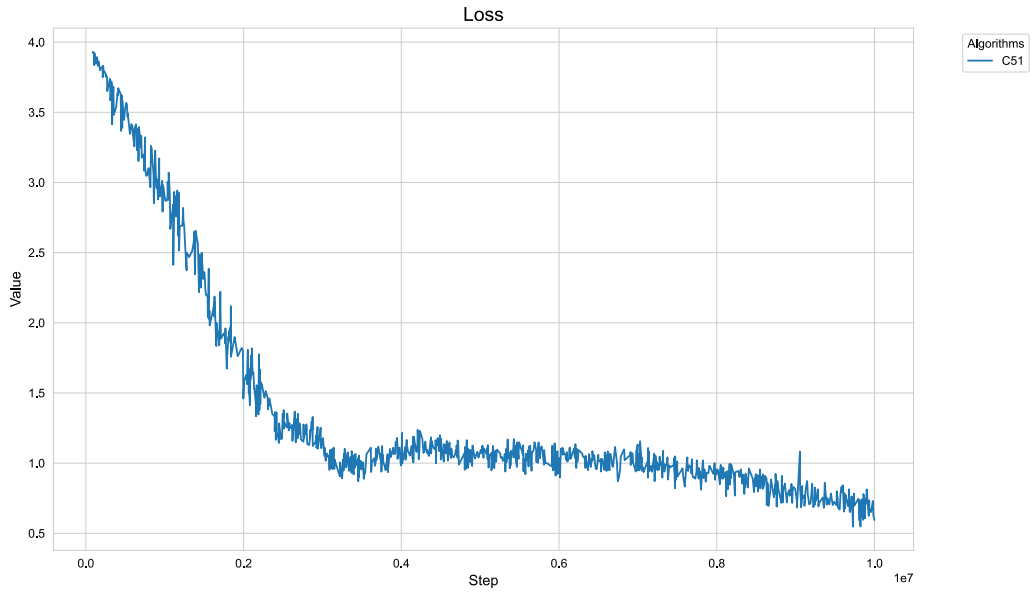

Fig. 12. Total loss: Shows the combined loss function value during training, incorporating all component losses.

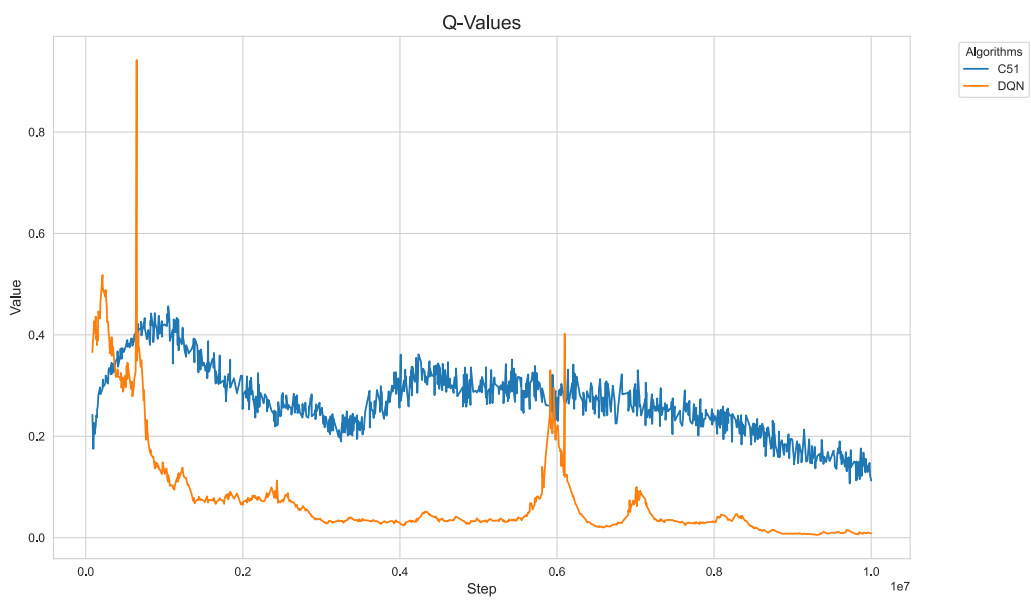

Fig. 13. Q-values: Shows the estimated action-value function outputs during training, demonstrating C51's superior performance compared to DQN.

for larger parameter updates early in training when the agent is far from optimal behavior. The gradual linear decrease to zero by the end of training helps fine-tune the policy more precisely, reducing the risk of overshooting optimal parameters. This balances exploration in early training (higher learning rates) and exploitation in later stages (lower learning rates). Unfortunately, the PPO agent did not find an appropriate combination of parameters in early training, which it could have later exploited, as discussed below.

The entropy pattern in Fig. 16 of the PPO agent provides insights into the agent's learning behavior. Starting with consistently high values around 1.6 in the early training phase, the entropy demonstrates considerable volatility throughout the process. This volatility suggests that while the agents maintain a high degree of exploration, it struggles to converge on effective strategies. The fluctuation between 0.8 and 1.6 over the 10 million training steps indicates a persistent uncertainty in action selection, rather than the gradual refinement we would expect to see in successful learning.

The approximate mean KL divergence of the PPO agent in Fig. 17 further demonstrates the high fluctuations in the training process with values ranging from near 0 to over 0.008 during training until the linearly declining learning rate is small enough to cause convergence. This sideways trend of the approximate mean KL divergence indicates that the agent is consistently making changes to its policy which get clipped, as indicated by Fig. 18, rather than converging towards a more defined strategy with less frequent and profound changes to the policy. The old approximate KL divergence shown in Fig. 19 exhibits similar volatility but with more extreme spikes reaching up to 0.020, particularly in the early stages around 0.2M steps. These dramatic spikes followed by periods of relative stability suggest that the agent occasionally made very large policy updates that considerably deviated from its previous behavior, but these major changes did not lead to sustained improvements in performance.

The policy loss shown in Fig. 20 exhibits persistent volatility throughout the training process, with values oscillating between approximately −0.015 and 0.010. This volatility indicates that the agent

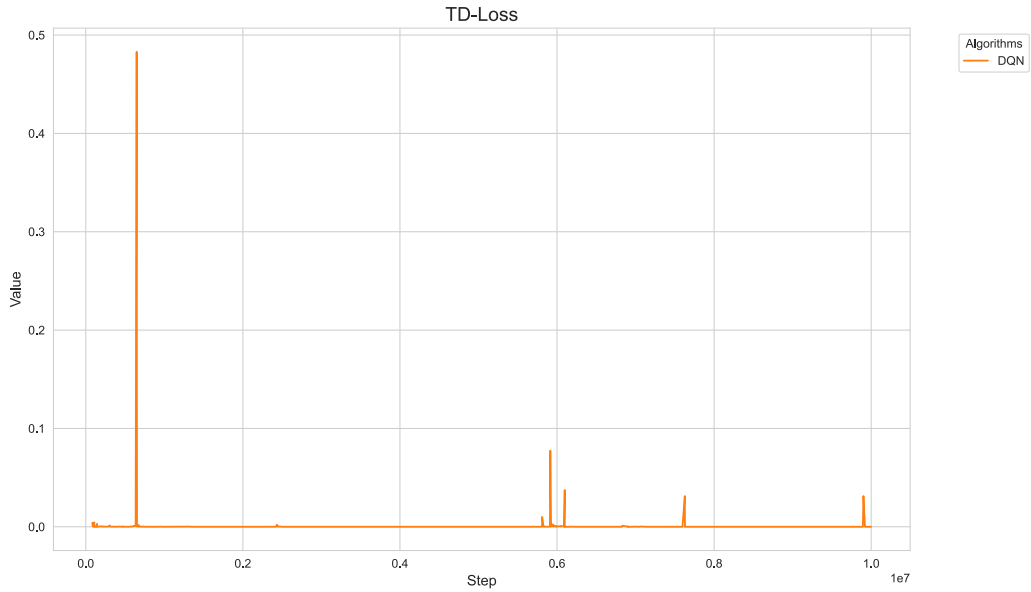

Fig. 14. Temporal difference loss: Represents the error in value prediction between consecutive states.

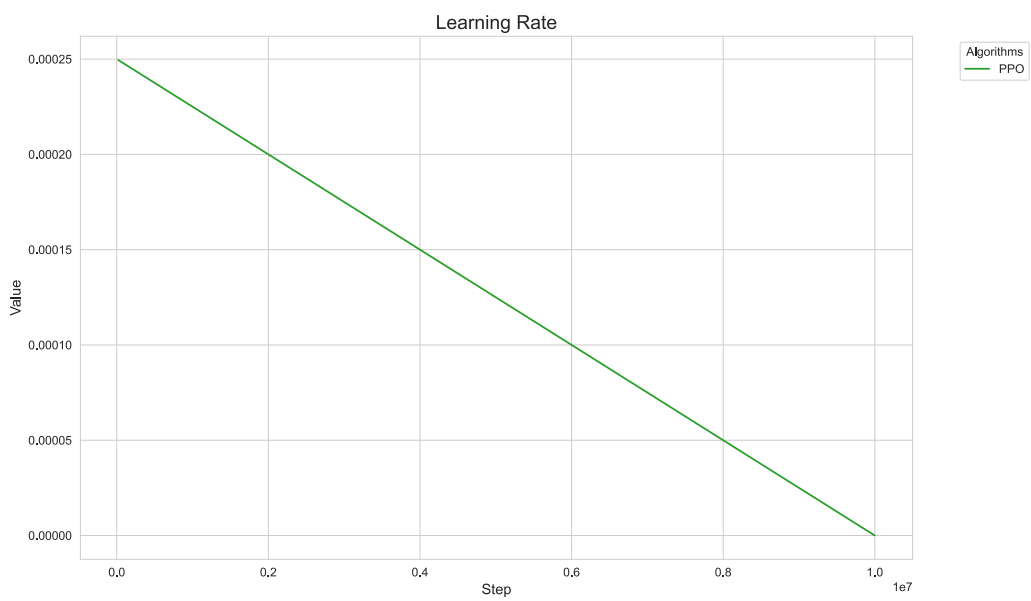

Fig. 15. Learning rate: Shows the adaptive learning rate schedule used in PPO, demonstrating the gradual decay pattern.

is making frequent and significant adjustments to its policy without achieving stable convergence. Notable spikes in policy loss around the 6M and 8M step marks demonstrate particularly unstable periods in the learning process. This pattern, combined with the previously discussed high entropy values and KL divergence, suggests that while the linear learning rate decay was intended to transition the agent from exploration to exploitation, the agent failed to identify and consolidate effective strategies. The continuous fluctuation in policy loss, even during later training stages where lower learning rates should promote more precise policy refinement, further supports the observation that the PPO agent struggled to develop a consistent and effective approach to the environment.

The value loss graph in Fig. 21 demonstrates a similar pattern to the policy loss with high volatility throughout the training process, with sporadic spikes reaching values up to 0.012. These sharp fluctuations, particularly prominent between 0.2M and 0.6M steps, suggest that the value network frequently encountered situations where its predictions deviated substantially from the actual returns. The pattern of intermittent spikes, rather than a gradual decrease in loss, indicates that the value network struggled to develop consistent and accurate value estimations. This unstable value prediction aligns with the agent's overall performance issues, as accurate value estimation is crucial for effective policy updates in PPO. While the baseline frequency of value loss remains relatively low, the persistent occurrence of error spikes even in later training stages suggests that the value network never achieved reliable convergence, contributing to the agent's inability to develop a stable and effective policy despite the implementation of learning rate decay.

The explained variance metric shown in Fig. 22 reveals further evidence of the agent's unstable learning process. The metric predominantly hovers near 0, with dramatic negative spikes reaching as low as −100 around the 0.2M step mark. These downward spikes, particularly in early training, indicate periods where the value network's predictions were especially poor — worse than simply predicting the mean value. The sustained pattern of values close to 0 throughout

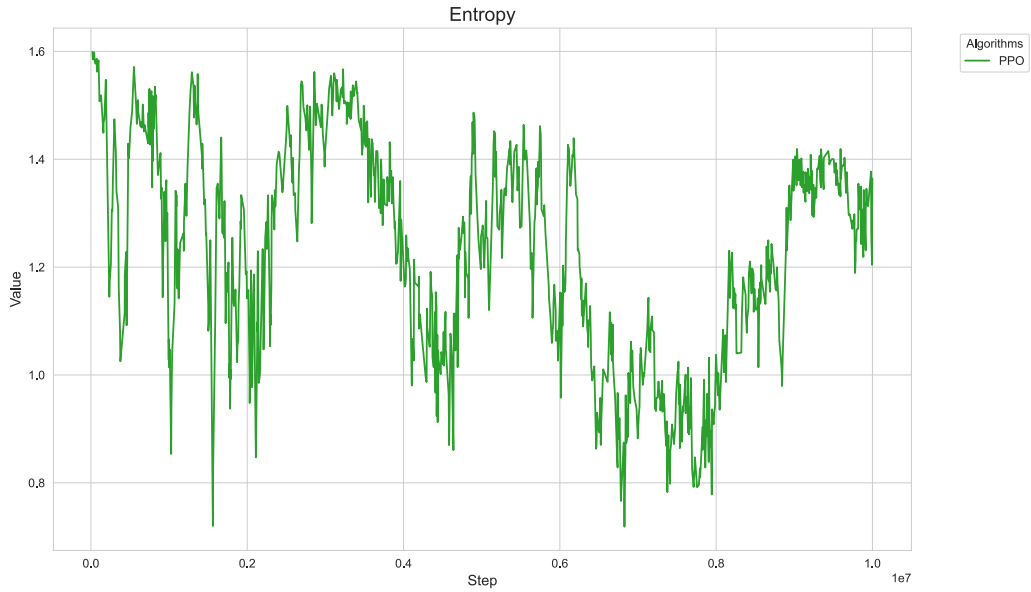

Fig. 16. Entropy: Represents the policy's exploration tendency, with higher values indicating more exploratory behavior.

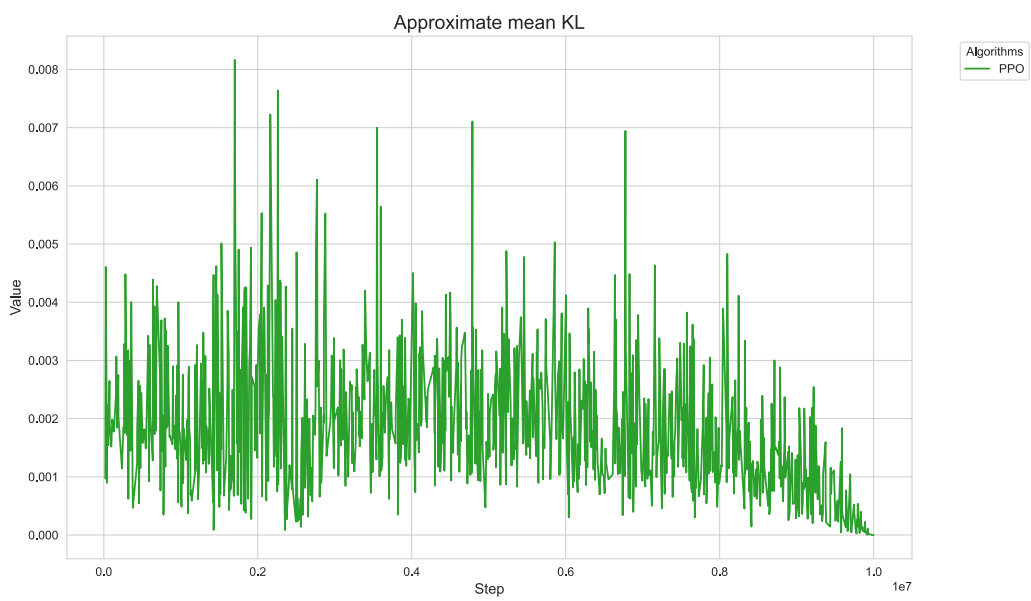

Fig. 17. Approximate KL divergence: Measures the difference between old and new policies during training updates.

training suggests that the value network struggled to capture meaningful relationships between states and returns, only managing to explain a minimal amount of variance in the observed rewards. This poor predictive performance aligns with the volatile value loss patterns and further explains the agent's difficulty in developing an effective policy.

PPO demonstrated unique advantages in training efficiency despite showing the lowest average episode length (562.1 steps). Using 8 parallel environment simulations, it completed the 10M steps in 17.45 h, less than half the time required by DQN (37.70 h) or C51 (39.24 h). Additionally, it was the only agent able to maintain stable steps per second (SPS), which makes PPO viable for training significantly more steps in a short amount of wall time.

5.3. Final remarks on the results

The SPS in Fig. 23 indicates that C51 is the most computationally expensive algorithm out of all three, with 70 SPS at 10 million steps. The relatively low SPS of all agents as shown in Fig. 23 indicates that the available computational resources of the Hummel standard nodes (Hardware-Konfiguration: RRZ, 2024) are not sufficient for simulating the ALE environment and training the agent efficiently. The standard nodes have 2 Intel Xeon Processor E5-2630 v3 (8 cores each) clocked at 2.4 GHz and no available CUDA devices.

The inability of the RL agent to learn to play ALE/Tetris-v5 within the given time steps of 10M can be attributed to various factors. Firstly, the image data provided by the environment to the CNN includes much redundant information, as the playfield is only a small part of the $(4 \times 84 \times 84)$ image provided, as can be seen in Fig. 4. Additionally, the reward in ALE/Tetris-v5 is very sparse, with the agent having to perform multiple actions to successfully clear a line. Furthermore, since a hard drop option is missing in ALE/Tetris-v5, the agent has long waiting times between lining up pieces and them hitting the ground, during which it explores other actions, potentially worsening the placement of the Tetriminoes.

In comparison to the RL agents, which not only required significantly more computational resources to train and evaluate, the heuristic

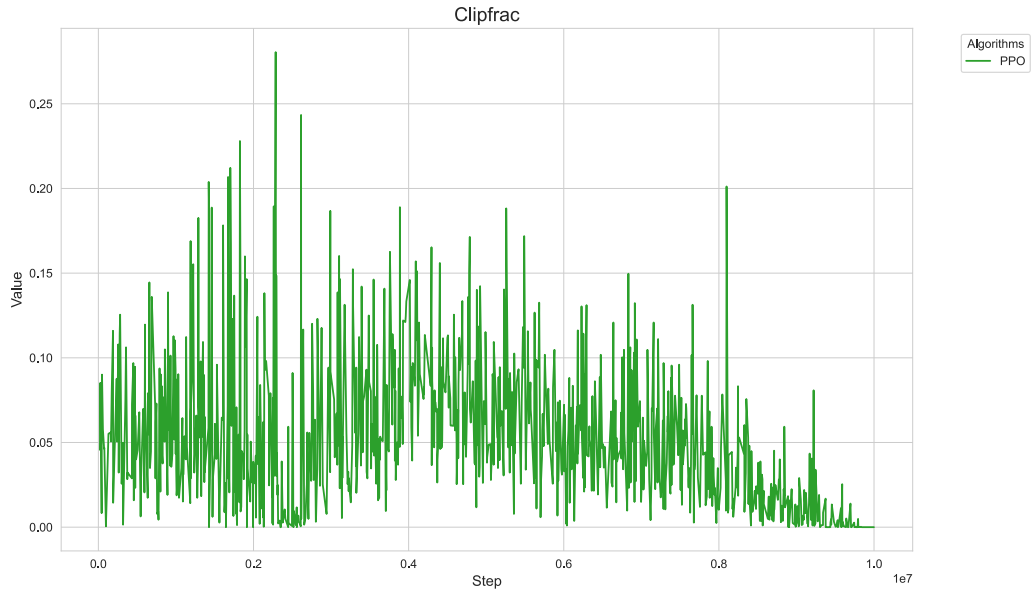

Fig. 18. Clip fraction: Indicates the proportion of policy updates that were clipped by PPO's optimization constraint.

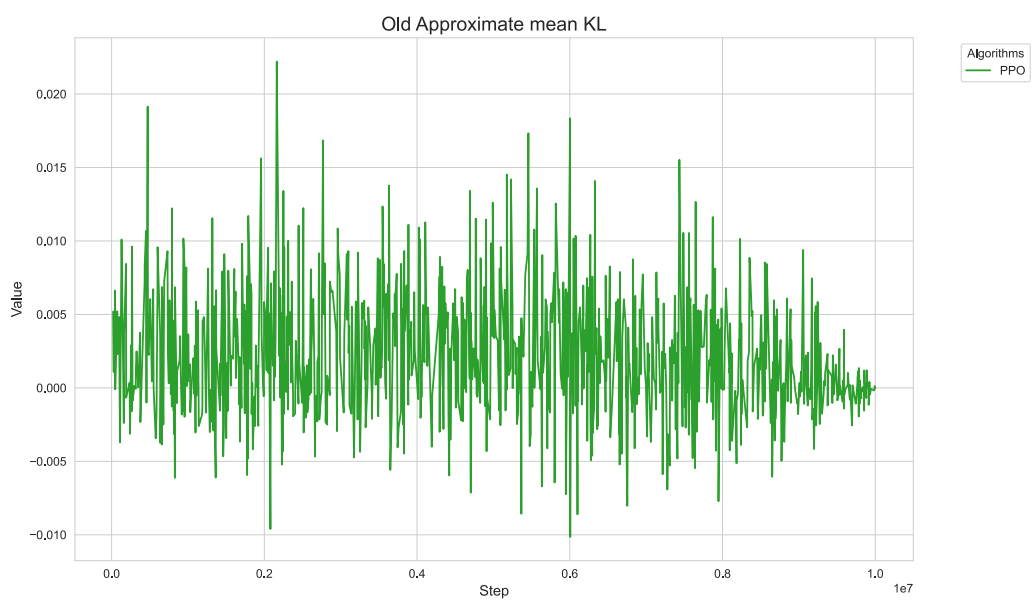

Fig. 19. Old approximate KL divergence: Tracks the historical KL divergence values between policy updates.

agent is also the only one able to be used in environments that are different from ALE/Tetris-v5. Only the location of the region of interest and potentially the threshold value for the binary representation would need to be altered to apply the agent to a different Tetris version.

While ALE/Tetris-v5 differs from modern Tetris variants, as detailed in Table 3, these differences actually strengthen rather than limit our findings. The absence of advanced mechanics like the 7-bag RNG, next piece preview, and hold feature creates a more challenging environment where agents must succeed without these advantages. This makes our comparison particularly relevant for understanding how RL and heuristic approaches perform under fundamental Tetris mechanics. The completely random piece generation in ALE/Tetris-v5 presents a more difficult challenge than the constrained randomness of the 7-bag system, as agents must develop robust strategies without guaranteed piece distributions. Furthermore, the simpler scoring system allows for clearer analysis of basic line-clearing capabilities without the complexity of modern scoring mechanics. Our results therefore provide valuable insights into the core strengths and limitations of different AI approaches in handling the essential elements of Tetris, which remain relevant regardless of specific game variants.

6. Conclusions

6.1. Summary of the study and its findings

Through the use of Tetris, we have shown the strengths, weaknesses, and applicability of RL and heuristic approaches in solving complex, real-time decision-making problems. The comparative analysis revealed

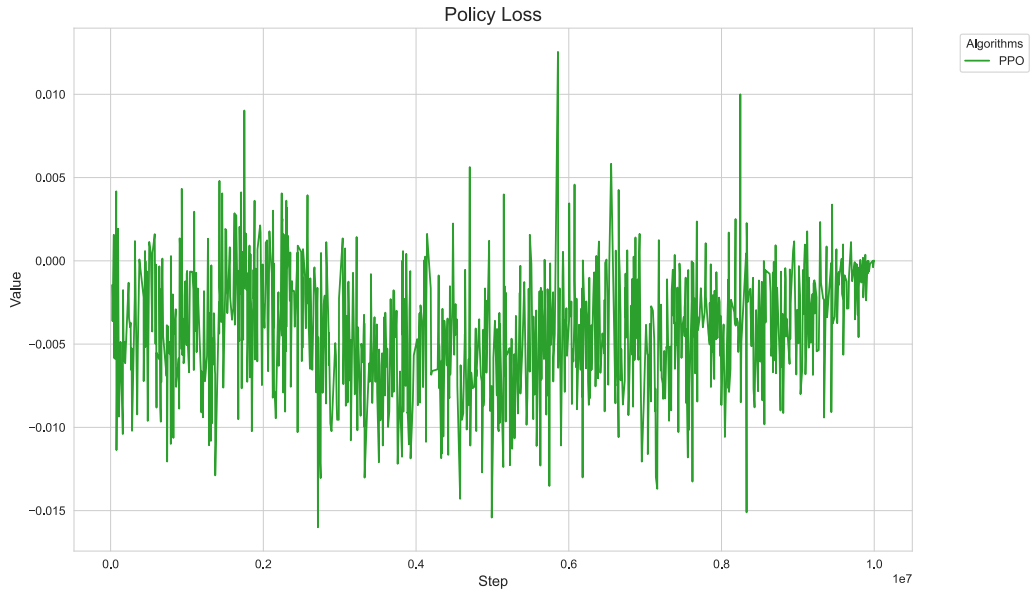

Fig. 20. Policy loss: Demonstrates the policy network's loss evolution during training.

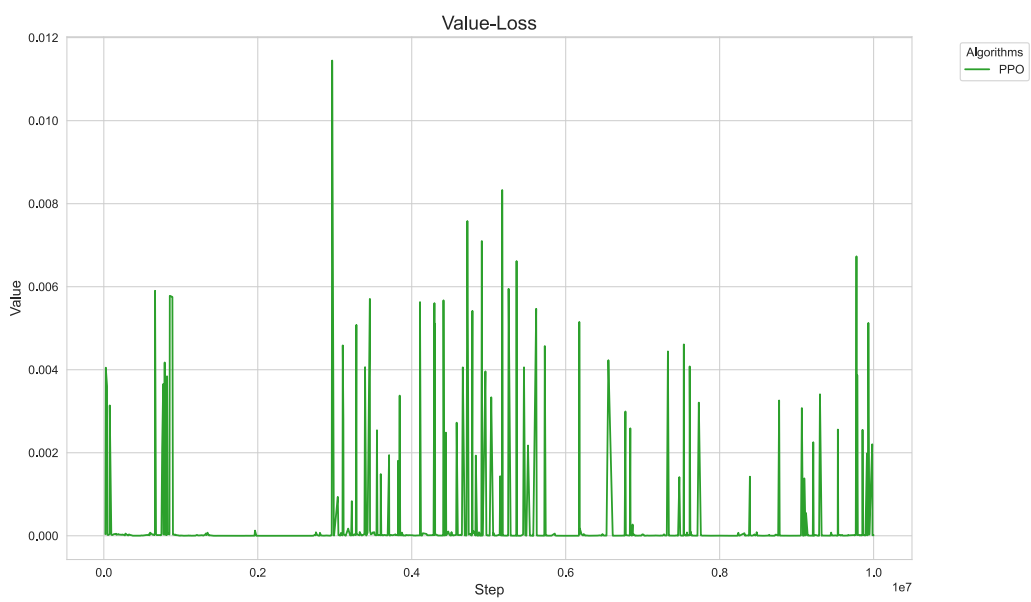

Fig. 21. Value loss: Shows the error in the value network's predictions throughout training.

considerable disparities in performance and efficiency among the methods. The heuristic agent was able to outperform the RL methods in both computational efficiency and achieved score.

The RL agents, despite employing SOTA algorithms such as C51 and PPO, struggled to learn effective strategies within the allotted 10 million steps. This underperformance can be explained by several factors, including the sparse reward structure of the ALE/Tetris-v5 environment, the absence of a hard drop feature, and the challenges posed by the high-dimensional input space. These findings underscore the difficulties RL faces in environments with delayed rewards and complex state spaces, particularly when learning from raw pixel inputs.

The study also revealed interesting differences between the RL algorithms tested. C51 showed more consistent positive rewards compared to DQN, suggesting benefits of its distributional approach. PPO, while not outperforming the other methods in terms of score, demonstrated superior computational efficiency by leveraging parallel environment simulations.

6.2. Limitations

The heuristic method demonstrated strengths in its ability to quickly make decisions based on predefined rules, showing particular efficacy in the Tetris environment where certain desirable board states can be easily defined. However, its limitation lies in its inability to adapt or discover novel strategies beyond its programmed rules.

The comparison was conducted using the ALE/Tetris-v5 environment, which differs in some aspects from standard Tetris implementations. This limits the direct comparability of the results to other Tetris variants. Additionally, the training duration for the RL agents was limited to 10 million steps. It is possible that with more training time, these agents could achieve better performance. However, using significantly more steps can be seen as not expedient as in cases where a large amount of computational resources can be employed, brute force methods such as BetaTetris by Wu (2024) are favorable. Furthermore, the study did not explore hyperparameter tuning, which could potentially improve the performance of the RL agents.

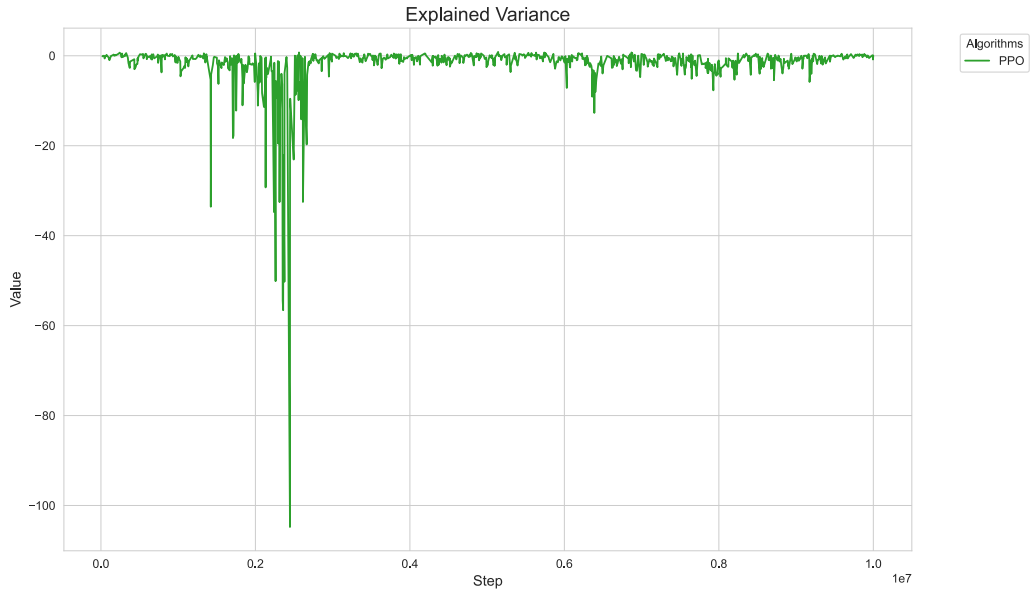

Fig. 22. Explained variance: Measures how well the value function predicts actual returns, indicating prediction accuracy.

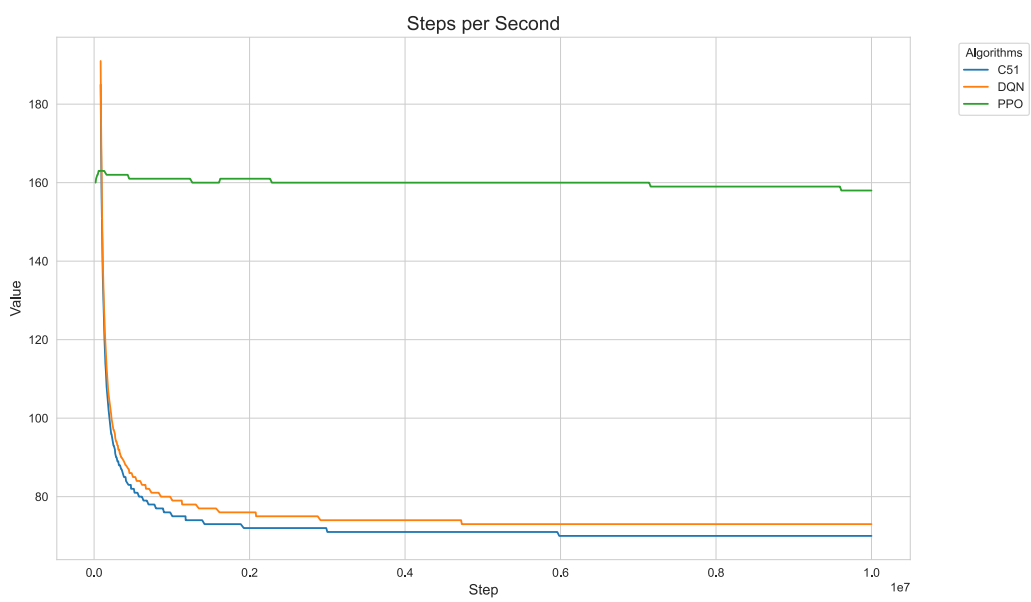

Fig. 23. Steps Per Second (SPS): Illustrates the computational efficiency of the algorithm throughout the training process.

6.3. Future outlook

This study provided a foundational framework for comparative analysis of AI approaches in complex gaming environments, which future research can build upon in several directions. The detailed examination of performance metrics, computational efficiency, and implementation challenges can serve as a baseline for evaluating new algorithms or hybrid approaches.

The findings suggest potential value in exploring dynamic or adaptive heuristic approaches that could combine the computational efficiency of traditional heuristics with some of the adaptability of RL methods. Such an approach could involve dynamically adjusting the weights of different heuristic components based on the current game state or recent performance metrics. For instance, the relative importance of hole prevention versus line clearing could be automatically tuned based on the current board height or the frequency of recent line clears. This could help address one of the key limitations of traditional heuristic approaches – their static nature – while maintaining their computational efficiency and interpretability. However, implementing such dynamic adjustments would need to be carefully balanced against the increased computational overhead they would introduce.

Moreover, extending the comparison to include other RL algorithms, such as Rainbow (Hessel et al., 2018), could provide a more comprehensive view of RL capabilities in this domain. Investigating the impact of different input representations for the RL agents might improve learning efficiency. Exploring the use of curriculum learning (Narvekar

et al., 2020) or other techniques to address the sparse reward problem in Tetris for RL agents could potentially overcome one of the key challenges identified in this study. Additionally, future work in applying DL methods for the ALE/Tetris-v5 environment should also investigate the feasibility of using Transformer models to play Tetris, similar to the work by Ruoss et al. (2024) for chess.

Finally, the integration of Support Vector Machines (SVMs) and Support Vector Regression (SVR) with RL can be a promising direction for future research. As demonstrated by An, Ding, Shi, and Li (2018), SVMs have strong capabilities in handling small sample, nonlinear and high-dimensional pattern recognition problems while avoiding issues like overfitting. Similarly, Ahmad, Adnan, Zaidi, and Bhargava (2020) showed SVR's effectiveness in predicting complex relationships with high accuracy and efficiency compared to other methods. These characteristics make SVM/SVR potentially valuable for addressing key challenges in RL, such as improving sample efficiency, handling continuous state–action spaces, and learning robust value function approximations. The evolutionary SVR approach proposed by Yeganeh, Abbasi, Shongwe, Malela-Majika, and Shadman (2024) further suggests that combining SVR with evolutionary optimization could enhance its performance in sequential decision-making tasks. Future work could explore using SVM/SVR as function approximators in RL algorithms, leveraging their strong generalization capabilities and ability to handle high-dimensional data. This could be particularly beneficial for real-world RL applications where data efficiency and robust learning are critical.

Acronyms

AI Artificial Intelligence
ALE Arcade Learning Environment
API Application Programming Interface
C51 Categorical DQN
CAP Credit Assignment Problem
CNN Convolutional Neural Network
DDQN Double Deep Q Network
DL Deep Learning
DNN Deep Neural Network
DQN Deep Q Network
DRL Deep Reinforcement Learning
FJS Flexible Job Shop Scheduling
FPS frames per second
GA Genetic Algorithm
GAE Generalized Advantage Estimation
GD Gradient Descent
IDS Intrusion Detection System
KL Kullback Leibler
LFSR Linear Feedback Shift Register
MDP Markov Decision Process
ML Machine Learning
MSE Mean Squared Error
NES Nintendo Entertainment System
NN Neural Network
NTSC National Television Standards Committee
PAL Phase Alternate Line
PMF Probability Mass Function
PPO Proximal Policy Optimization
Q Quality
ReLU Rectified Linear Unit
RL Reinforcement Learning
RNG Random Number Generator
SARSA State Action Reward State Action
SGD Stochastic Gradient Descent
SOTA state-of-the-art
SPS steps per second
SRS Super Rotation System
SVM Support Vector Machine
SVR Support Vector Regression
TD Temporal Difference

CRediT authorship contribution statement

Julius A. Bairaktaris: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Visualization. **Arne Johannssen:** Conceptualization, Methodology, Formal analysis, Investigation, Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank Iztok Fister Jr. and three anonymous reviewers for their valuable feedback and suggestions, which were very important and helpful to significantly improve the paper.

Data availability

No data was used for the research described in the article.

References

Abusitta, A., Li, M. Q., & Fung, B. C. (2024). Survey on explainable AI: techniques, challenges and open issues. *Expert Systems with Applications*, *255*, Article 124710. http://dx.doi.org/10.1016/j.eswa.2024.124710.

Ahmad, M. S., Adnan, S. M., Zaidi, S., & Bhargava, P. (2020). A novel support vector regression (SVR) model for the prediction of splice strength of the unconfined beam specimens. *Construction and Building Materials*, *248*, Article 118475. http://dx.doi.org/10.1016/j.conbuildmat.2020.118475, https://www.sciencedirect.com/science/article/pii/S0950061820304803.

An, Y., Ding, S., Shi, S., & Li, J. (2018). Discrete space reinforcement learning algorithm based on support vector machine classification. *Pattern Recognition Letters*, *111*, 30–35. http://dx.doi.org/10.1016/j.patrec.2018.04.012, https://www.sciencedirect.com/science/article/pii/S0167865518301326.

Arora, S., Cohen, N., Golowich, N., & Hu, W. (2018). A convergence analysis of gradient descent for deep linear neural networks. CoRR, abs/1810.02281.

Bai, H., Cheng, R., & Jin, Y. (2023). Evolutionary reinforcement learning: A survey. *Intelligent Computing*, *2*, http://dx.doi.org/10.34133/icomputing.0025.

Bellemare, M. G., Candido, S., Castro, P. S., Gong, J., Machado, M. C., Moitra, S., et al. (2020). Autonomous navigation of stratospheric balloons using reinforcement learning. *Nature*, *588*(7836), 77–82. http://dx.doi.org/10.1038/s41586-020-2939-8.

Bellemare, M. G., Dabney, W., & Munos, R. (2017). A distributional perspective on reinforcement learning. In *ICML'17: vol. 70, Proceedings of the 34th international conference on machine learning* (pp. 449–458). Sydney, NSW, Australia: JMLR.org.

Bellemare, M. G., Naddaf, Y., Veness, J., & Bowling, M. (2013). The arcade learning environment: an evaluation platform for general agents. *Journal of Artificial Intelligence Research*, *47*, 253–279.

Bellman, R., & Dreyfus, S. (2010). Dynamic programming, *Princeton Landmarks in Mathematics ed., with a new introduction*, (1st ed.). Princeton, NJ: Princeton University Press.

Bose, S. R., & Kumar, V. S. (2022). In-situ recognition of hand gesture via Enhanced Xception based single-stage deep convolutional neural network. *Expert Systems with Applications*, *193*, Article 116427. http://dx.doi.org/10.1016/j.eswa.2021.116427.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., et al. (2018). JAX: composable transformations of python+numpy programs. http://github.com/google/jax.

Brunton, S. L. (2021). Machine learning meets control theory. http://dx.doi.org/10.52843/cassyni.x2t0sp.

Brunton, S. L. (2022). Overview of deep reinforcement learning methods. http://dx.doi.org/10.52843/cassyni.kfnzpy.

Brunton, S. L., & Kutz, J. N. (2022). *Data-driven science and engineering: machine learning, dynamical systems, and control* (2nd ed.). Cambridge University Press.

Campbell, M., Hoane, A. J., & Hsu, F.-h. (2002). Deep blue. *Artificial Intelligence*, *134*(1), 57–83. http://dx.doi.org/10.1016/S0004-3702(01)00129-1.

Cannon, G. (2024). StackRabbit: An AI for playing NES tetris at a high level. Based primarily on search and heuristic, with high quality board evaluation through value iteration.

Carapuço, J., Neves, R., & Horta, N. (2018). Reinforcement learning applied to forex trading. *Applied Soft Computing*, *73*, 783–794. http://dx.doi.org/10.1016/j.asoc.2018.09.017.

Castro, P. S., Moitra, S., Gelada, C., Kumar, S., & Bellemare, M. G. (2018). Dopamine: a research framework for deep reinforcement learning.

Choi, D., Shallue, C. J., Nado, Z., Lee, J., Maddison, C. J., & Dahl, G. E. (2019). On empirical comparisons of optimizers for deep learning. CoRR, abs/1910.05446.

Chouard, T. (2016). The go files: AI computer wraps up 4-1 victory against human champion. *Nature*, http://dx.doi.org/10.1038/nature.2016.19575.

Clune, J. (2007). Heuristic evaluation functions for general game playing. *vol. 7*, In *AAAI* (pp. 1134–1139).

Contributors, T. (2024). TetrisGYM: NES Tetris training ROM. https://github.com/kirjavascript/TetrisGYM, Training ROM modification for NES Tetris with practice features.

Dann, C., Mansour, Y., Mohri, M., Sekhari, A., & Sridharan, K. (2022). Guarantees for Epsilon-Greedy reinforcement learning with function approximation.

Ding, Z., Huang, Y., Yuan, H., & Dong, H. (2020). Introduction to reinforcement learning. In H. Dong, Z. Ding, & S. Zhang (Eds.), *Deep reinforcement learning: fundamentals, research and applications* (pp. 47–123). Springer Singapore, http://dx.doi.org/10.1007/978-981-15-4095-0.

Dong, H., Ding, Z., & Zhang, S. (2020). *Deep reinforcement learning: fundamentals, research and applications* (1st ed.). (p. xxvii, 514). Springer Singapore, http://dx.doi.org/10.1007/978-981-15-4095-0.

Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., et al. (2020). Implementation matters in deep RL: A case study on PPO and TRPO. In *International conference on learning representations*.

Gourion, D. (2021). An upper bound for the number of chess diagrams without promotion.

Hardware-Konfiguration: RRZ (2024). https://www.rrz.uni-hamburg.de/services/hpc/hummel-2015/hardware-konfiguration.html. (Accessed 22 August 2024).

van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *30*(1), http://dx.doi.org/10.1609/aaai.v30i1.10295.

Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., et al. (2024). Flax: A neural network library and ecosystem for JAX. http://github.com/google/flax.

Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., et al. (2018). Rainbow: combining improvements in deep reinforcement learning. In *AAAI'18/IAAI'18/EAAI'18, Proceedings of the thirty-second AAAI conference on artificial intelligence and thirtieth innovative applications of artificial intelligence conference and eighth AAAI symposium on educational advances in artificial intelligence*. New Orleans, Louisiana, USA: AAAI Press.

hexhowells (2022). Superstacker. https://github.com/hexhowells/SuperStacker.

Houssein, E. H., Hassaballah, M., Ibrahim, I. E., AbdElminaam, D. S., & Wazery, Y. M. (2022). An automatic arrhythmia classification model based on improved marine predators algorithm and convolutions neural networks. *Expert Systems with Applications*, *187*, Article 115936. http://dx.doi.org/10.1016/j.eswa.2021.115936.

Huang, S., Dossa, R. F. J., Raffin, A., Kanervisto, A., & Wang, W. (2022). The 37 implementation details of proximal policy optimization. In *ICLR blog track*. https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/.

Huang, S., Dossa, R. F. J., Ye, C., Braga, J., Chakraborty, D., Mehta, K., et al. (2022). CleanRL: high-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, *23*(274), 1–18.

Huynh, C. (2020). tetrisAI: deep Q-learning network approach to teach a reinforcement learning agent how to play Tetris. https://github.com/ChesterHuynh/tetrisAI.

Hydrant, Vernie, meme, Eren, Huffulufugus, PixelAndy, et al. (2024). NES tetris leaderboards. https://docs.google.com/spreadsheets/d/1ZBxkZEsfwDsUpyire4Xb16er36Covk7nhR8BN_LPodI/edit?pli=1&gid=2002358653#gid=2002358653, (Accessed 22 August 2024).

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Y. Bengio, & Y. LeCun (Eds.), *3rd international conference on learning representations, ICLR 2015, san diego, CA, USA, May 7-9, 2015, conference track proceedings*.

Korf, R. E. (1990). Real-time heuristic search. *Artificial Intelligence*, *42*(2), 189–211. http://dx.doi.org/10.1016/0004-3702(90)90054-4.

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324. http://dx.doi.org/10.1109/5.726791.

Lewis, I., & Beswick, S. (2015). Generalisation over details: the unsuitability of supervised backpropagation networks for tetris. *Advances in Artificial Neural Systems*, *2015*, 1–8. http://dx.doi.org/10.1155/2015/157983.

Lopez-Martin, M., Carro, B., & Sanchez-Esguevillas, A. (2020). Application of deep reinforcement learning to intrusion detection for supervised problems. *Expert Systems with Applications*, *141*, Article 112963. http://dx.doi.org/10.1016/j.eswa.2019.112963.

Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, *91*, Article 106208. http://dx.doi.org/10.1016/j.asoc.2020.106208.

Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M. J., & Bowling, M. (2018). Revisiting the arcade learning environment: evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, *61*, 523–562.

McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, *102*(3), 419–457. http://dx.doi.org/10.1037/0033-295X.102.3.419.

Minsky, M. (1961). Steps toward artificial intelligence. *Proceedings of the IRE*, *49*(1), 8–30. http://dx.doi.org/10.1109/JRPROC.1961.287775.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533. http://dx.doi.org/10.1038/nature14236.

Mott, B. W., Anthony, S., & Team, T. S. (2024). Stella: A multi-platform atari 2600 VCS emulator. https://stella-emu.github.io/.

Muduli, R., Jena, D., & Moger, T. (2024). A survey on load frequency control using reinforcement learning-based data-driven controller. *Applied Soft Computing*, *166*, Article 112203. http://dx.doi.org/10.1016/j.asoc.2024.112203.

Müller-Brockhausen, M., Preuss, M., & Plaat, A. (2021). A new challenge: approaching tetris link with AI. In *2021 IEEE conference on games (coG)* (pp. 1–8). http://dx.doi.org/10.1109/CoG52621.2021.9619044.

Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., De Maria, A., et al. (2015). Massively parallel methods for deep reinforcement learning. ArXiv, abs/1507.04296.

Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M. E., & Stone, P. (2020). Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, *21*(181), 1–50, http://jmlr.org/papers/v21/20-212.html.

Ng, A., Harada, D., & Russell, S. J. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In *International conference on machine learning*.

OpenAI (2018). A taxonomy of RL algorithms. https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html#a-taxonomy-of-rl-algorithms. (Accessed 29 July 2024).

Park, J., Seo, Y., Shin, J., Lee, H., Abbeel, P., & Lee, K. (2022). SURF: semi-supervised reward learning with data augmentation for feedback-efficient preference-based reinforcement learning. In *International conference on learning representations*.

Phon-Amnuaisuk, S. (2015). Evolving and discovering tetris gameplay strategies. *Procedia Computer Science*, *60*, 458–467. http://dx.doi.org/10.1016/j.procs.2015.08.167, https://www.sciencedirect.com/science/article/pii/S1877050915022942, Knowledge-Based and Intelligent Information & Engineering Systems 19th Annual Conference, KES-2015, Singapore, September 2015 Proceedings.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: reliable reinforcement learning implementations. *Journal of Machine Learning Research*, *22*(268), 1–8.

Ruder, S. (2017). An overview of gradient descent optimization algorithms.

Ruoss, A., Delétang, G., Medapati, S., Grau-Moya, J., Wenliang, L. K., Catt, E., et al. (2024). Grandmaster-level chess without search.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., et al. (2020). Mastering Atari, go, chess and shogi by planning with a learned model. *Nature*, *588*(7839), 604–609. http://dx.doi.org/10.1038/s41586-020-03051-4.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms.

Schultz, W. (2004). Neural coding of basic reward terms of animal learning theory, game theory, microeconomics and behavioural ecology. *Current Opinion in Neurobiology*, *14*(2), 139–147. http://dx.doi.org/10.1016/j.conb.2004.03.017.

Shakya, A. K., Pillai, G., & Chakrabarty, S. (2023). Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, *231*, Article 120495. http://dx.doi.org/10.1016/j.eswa.2023.120495.

Shen, S., Cai, C., Li, Z., Shen, Y., Wu, G., & Yu, S. (2024). Deep Q-network-based heuristic intrusion detection against edge-based SIoT zero-day attacks. *Applied Soft Computing*, *150*, Article 111080. http://dx.doi.org/10.1016/j.asoc.2023.111080.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., et al. (2017). Mastering the game of go without human knowledge. *Nature*, *550*(7676), 354–359. http://dx.doi.org/10.1038/nature24270.

Soler, D., Mariño, O., Huergo, D., de Frutos, M., & Ferrer, E. (2024). Reinforcement learning to maximize wind turbine energy generation. *Expert Systems with Applications*, *249*, Article 123502. http://dx.doi.org/10.1016/j.eswa.2024.123502.

Tetris Wiki - Playing Forever (2024).

Tetris wiki - SRS (2024). https://tetris.fandom.com/wiki/SRS. (Accessed 10 August 2024).

Tetris Wiki - Tetris Guideline (2024). https://tetris.fandom.com/wiki/Tetris_Guideline. (Accessed 10 August 2024).

The Tetris Company (2024). History of tetris. https://tetris.com/history-of-tetris. (Accessed 10 August 2024).

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, *575*(7782), 350–354. http://dx.doi.org/10.1038/s41586-019-1724-z.

Weng, J., Chen, H., Yan, D., You, K., Duburcq, A., Zhang, M., et al. (2022). Tianshou: A highly modularized deep reinforcement learning library. *Journal of Machine Learning Research*, *23*(267), 1–6.

Wu, A. (2024). BetaTetris tablebase. https://github.com/adrien1018/betatetris-tablebase.

Yeganeh, A., Abbasi, S. A., Shongwe, S. C., Malela-Majika, J.-C., & Shadman, A. R. (2024). Evolutionary support vector regression for monitoring Poisson profiles. *Soft Computing*, *28*(6), 4873–4897. http://dx.doi.org/10.1007/s00500-023-09047-2.

Zhang, P., Wang, J., Tian, Z., Sun, S., Li, J., & Yang, J. (2022). A genetic algorithm with jumping gene and heuristic operators for traveling salesman problem. *Applied Soft Computing*, *127*, Article 109339. http://dx.doi.org/10.1016/j.asoc.2022.109339.

Zhou, G., Tian, W., Buyya, R., & Wu, K. (2023). Growable genetic algorithm with heuristic-based local search for multi-dimensional resources scheduling of cloud computing. *Applied Soft Computing*, *136*, Article 110027. http://dx.doi.org/10.1016/j.asoc.2023.110027.